

Master Thesis

Rapid FPGA Verification Strategies for Servo Drives Using MPSoC

A thesis submitted for the degree of

Master of Science (M.Sc.)

to Darmstadt University of Applied Sciences

Department of Electrical Engineering and Information
Technology

submitted by

Suhas Bangalore Mruthunjaya

Matr. Nr. 1119281

Completion date : 30-12-2025

First Supervisor : Prof. Dr.-Ing. Thomas Schumann

Second supervisor : Prof. Dr. Georgi Nikolov

Company Supervisor : André Berger, Marcel Koehler

Student

Suhas

Bangalore Mruthunjaya

First (Given) Name

Last (Family) Name

22-09-1998

1119281

Date of Birth

Matr.-No.

Prof. Dr.-Ing. Thomas Schumann

1st Academic Supervisor

Prof. Dr. Georgi Nikolov

2nd Academic Supervisor

In partial fulfillment of the requirements of the **University of Applied Sciences Hochschule Darmstadt (h_da)** for the degree **Master of Science in Electrical Engineering** carried out in collaboration with **Industrial Enterprise**

Company: **Schneider Electric Automation GmbH**

Address: **Schneiderplatz 1 • 97828 Marktheidenfeld**

- ✓ This Master Thesis contains confidential data and may only be made available to the supervisor, the members of the examination board and authorized members of Darmstadt University of Applied Sciences.

Declaration

I hereby declare that this thesis is a presentation of my original research work and that no other sources were used other than what is cited.

I furthermore declare that wherever contributions of others are involved, this contribution is indicated, clearly acknowledged and due reference is given to the author and source.

I also certify that all content without reference or citation contained in this thesis is original work.

I acknowledge that any misappropriation of the previous declarations can be considered a case of academic fraud.

Würzburg, 30-12-2025

A handwritten signature in blue ink, reading "Suhas B.M.", is displayed within a light gray rectangular box.

Suhas Bangalore
Mruthunjaya

Abstract

This thesis introduces a proof-of-concept FPGA Verification Platform designed to enhance and accelerate the verification and validation of FPGA modules within servo drive control boards at Schneider Electric. Existing verification processes rely on manual procedures, static tests, and external hardware such as motors, resulting in extended development cycles, limited test coverage, and difficulties observing timing-critical FPGA modules. To overcome these challenges, a dedicated platform was developed utilizing a Zynq UltraScale+ MPSoC-based Enclustra Mercury XU5 System-on-Module and a DAC80508 evaluation board. This platform enables automated, repeatable, and independent testing of key FPGA modules, including the Current Sense SPI interface, PWM generator, and Overcurrent (OVC) detection module, as well as evaluation of module responses on actual hardware.

The platform supports both static and dynamic testing, utilizes software control through AXI interfaces, and integrates on-chip signal analysis for real-time verification. Experimental results demonstrate that the platform accurately reproduces reference signals, reliably detects current thresholds, and consistently measures PWM performance under diverse conditions. These findings indicate that the platform reduces manual intervention, facilitates fault detection, and enables comprehensive validation prior to release without the need for external hardware. This work establishes a foundation for the future development of a fully automated continuous integration and continuous deployment (CI/CD) verification platform for servo-drive FPGA designs.

Contents

List of Figures	v
List of Tables	vi
Acronyms	vii
I Thesis	1
1 Introduction	2
1.1 Motivation	2
1.2 Company Background	3
1.2.1 Servomechanism	3
1.2.2 Servo Drives	4
1.3 Problem Statement	5
1.4 Desired Outcome	6
2 Technical background	8
2.1 Zynq UltraScale+ MPSoC FPGA	8
2.2 Clock Nuances in FPGA-Design	10
2.3 Types of FPGA Verification	12
2.3.1 Simulation Verification	14
2.3.2 In-System Verification	16
2.3.3 Software-Driven Verification	17
2.3.4 Hardware-in-loop Verification	18
2.4 System Integrated Logic Analyzer	18
2.5 AMBA AXI Interfaces	20
3 Concept and Modeling	25
3.1 Servo Drive Control Board Architecture	25
3.2 Understanding the DUT	29
3.2.1 CurrentSense SPI Path	31

3.2.2	OVC Detect	32
3.2.3	PWM Control	33
3.3	Requirement Analysis	34
3.3.1	Enclustra Mercury XU5 + PE1 Evaluation board	35
3.3.2	DAC80508 Evaluation Board	37
3.3.3	Block Design of the proposed FPGA Verification Platform (Proof of concept)	42
3.4	Related Work and Case studies in FPGA and HIL Verification	43
3.4.1	PoC Solutions to Identified Research Gaps	46
4	Implementation	48
4.1	Current Sense Verification	48
4.1.1	Custom MUX algorithmn	50
4.1.2	Direct Digital Frequency Synthesis	52
4.1.3	Clock period selection for SPI	55
4.2	PWM Verification	56
4.2.1	PWM_IP Inputs and Control Commands	56
4.2.2	Block Design of PWM Verification	59
4.2.3	PWM Analyzer	60
4.2.4	Dead Time Capture	63
4.3	OVC Verification	64
4.3.1	Block Design of OVC Verification	66
4.4	Processing Subsystem application	68
4.4.1	Communication and Data Handling Layer	68
4.4.2	Verification Controller fsm	69
4.4.3	Signal Acquisition and Spectral Analysis	72
4.4.4	Pulse-based Signal Evaluation	76
4.4.5	Protection Event Monitoring	79
5	Results and Validation	82
5.1	Current Sense Verification Results	82
5.1.1	Static Test Results	82
5.1.2	Dynamic Test Results	83

5.1.3	Reference Values vs measured Values	84
5.2	PWM Verification results	86
5.3	OVC Verification Results	87
6	Conclusion	88
6.1	Summary	88
6.2	Future Work	90
II	Appendix	91
A	16 kHz PWM With Variable Duty Cycles	92
	Bibliography	93

List of Figures

1.1	Lexium 32 Servo Drive and Servo motor from Schneider Electric [12] . . .	4
1.2	Control Board Component Diagram	7
2.1	The Xilinx UltraScale MPSoC architecture [18]	10
2.2	Clocking Wizard Block Diagram [21]	11
2.3	An example illustrating the use of Clocking Wizard and Xilinx CDC IP Core	13
2.4	Types of FPGA Simulation Verification	14
2.5	System ILA IP Core	16
2.6	An example for AXI4 IP Core	17
2.7	System ILA Symbol [20]	19
2.8	System ILA capture of AXI4 write transaction signals.	20
2.9	The figure illustrates the operation of a write transaction, which utilizes the write request, write data, and write response channels [4].	22
2.10	The figure depicts a read transaction, demonstrating the use of the read request and read data channels [4].	22
3.1	PacDrive 3 Platform. ServoDrive - System Architecture	26
3.2	Architectural division of Servo Drive Control Board	27
3.3	Internal architecture of the servo drive showing separation between control (cold side) and power (hot side) electronics	28
3.4	Control Board MPSoC View	30
3.5	Simplified block representation of the three-phase current-sensing chain within the control board	32
3.6	Simplified block representation of the Over-Current Detect path within the control board	33
3.7	Simplified block representation of the PWM Generation Module within the control board	34
3.8	Mercury XU5 SoM Top View Module [14]	36
3.9	Mercury PE1 Board Top View [15]	36

3.10	DAC80508-04EVM Evaluation board Top View [22]	38
3.11	Block Design of the FPGA Verification Platform PoC	43
4.1	Block Design of the Current Sense Verification	50
4.2	Current MUX algorithm	51
4.3	Direct Digital frequency Synthesis	52
4.4	Sine wave generated from the DDFS	55
4.5	The PWM Signals generated at the CB with variable duty-cycle values	59
4.6	Block diagram depicting the closed-loop PWM verification architecture, which demonstrates the interaction among the control board, SoM, and FPGA based signal analysis modules	60
4.7	PWM Analyzer RTL Block Diagram	61
4.8	Dead Time Capture RTL Block Diagram	64
4.9	Block diagram of the Overcurrent Protection FPGA Implementation in the Control Board	65
4.10	Block Design of the Overcurrent Verification setup: Note that apart from the DDFS, the same analog path is reused for OVC Verification	67
4.11	Verification Controller FSM	71
5.1	Current Sense Static Verification Results	83
5.2	Reconstructed time domain representation and FFT of the feedback signal in matlab	83
5.3	Static Reference Voltage Values VS CB feedback Voltage Values	84
5.4	Current Sense Dynamic Test Results	85
5.5	FFT comparison of the reference 1 kHz digital signal and the feedback received from the Control Board.	85
5.6	PWM Verification Results	86
5.7	Overcurrent module Verification Results	87

List of Tables

3.1	CONFIG Register (Address = 0x03, Reset = 0x0000) [23]	40
3.2	GAIN Register (Address = 0x04) [23]	41
3.3	STATUS Register (Address = 0x07, Reset = 0x0000) [23]	41
3.4	DACx Registers (Addresses = 0x08 to 0x0F) [23]	41
4.1	PWM frequency settings	57
4.2	Overview of Control Commands and Corresponding Verification Functions	70

Acronyms

AC	Alternating Current
ADC	Analog-to-Digital Converter
AMBA	Advanced Microcontroller Bus Architecture
AXI	Advanced eXtensible Interface
BMP	Brandlabel Motors & Powerstages GmbH
CB	Control Board
CDC	Clock Domain Crossing
CMSIS-DSP	Cortex Microcontroller Software Interface Standard – Digital Signal Processing
CRC	Cyclic Redundancy Check
DAC	Digital-to-Analog Converter
DC	Direct Current
DDFS	Direct Digital Frequency Synthesis
DDR4 SDRAM	Double Data Rate 4 Synchronous Dynamic Random-Access Memory
DUT	Device Under Test
FIFO	First-In, First-Out
FIL	FPGA-in-the-Loop
FSM	Finite State Machine
FPGA	Field-Programmable Gate Array
FFT	Fast Fourier Transform
GPIO	General-Purpose Input/Output
HIL	Hardware-in-the-Loop
IEEE	Institute of Electrical and Electronics Engineers
IGBT	Insulated-Gate Bipolar Transistor
ILA	Integrated Logic Analyzer
IP	Intellectual Property
JTAG	Joint Test Action Group
LUT	Look-Up Table
MPSoC	Multiprocessor System-on-Chip

OVC	Overcurrent
PCIe	Peripheral Component Interconnect Express
PL	Programmable Logic
PoC	Proof of Concept
PS	Processing System
PWM	Pulse-Width Modulation
SoC	System-on-Chip
SPI	Serial Peripheral Interface
TSN	Time-Sensitive Network
UART	Universal Asynchronous Receiver/Transmitter

Part I

Thesis

1 Introduction

This chapter introduces the motivation, background, and context for developing an FPGA verification platform for servo drive control boards. It offers an overview of Schneider Electric's technological environment, the function of servomechanisms, the design of the servo drive control board, and the limitations inherent in the current verification process.

1.1 Motivation

Modern FPGA design and development rely on thorough verification to ensure safe, accurate, and efficient system behavior. At Schneider Electric, every new servo drive control board using an MPSoC incorporates updated or redesigned FPGA modules, which must be validated under realistic operating conditions before release. However, the existing verification process presents several limitations, resulting in prolonged development cycles, limited test coverage, delayed product releases, and an increased risk of critical issues remaining undetected until later stages.

To address these challenges, the company requires a dedicated FPGA Verification Platform capable of emulating necessary inputs, automating test sequences, and enabling dynamic, repeatable, and independent verification of FPGA modules. Such a platform would eliminate reliance on external components, offer precise control over input signals, reduce verification time, and ensure consistent test results across multiple runs.

This project seeks to modernize and enhance the FPGA verification process by developing a flexible, scalable, and automated platform for comprehensive real-hardware testing. By treating each FPGA module as a black box and generating precise digital and analog inputs, the platform addresses existing inefficiencies. The implementation will enable Schneider Electric to accelerate development, increase reliability, and ensure that new control boards comply with stringent performance and safety standards.

1.2 Company Background

Schneider Electric Automation GmbH is a global leader in energy management and automation. Since its founding in the 19th century, the company has expanded to offer solutions in industrial automation, building management, and sustainable energy. Its advanced servo systems enhance precision and energy efficiency in robotics, factory automation, and CNC machinery. Schneider Electric continues to play a key role in advancing servomechanism technology and robotics.

Schneider Electric's innovations in industrial automation and robotics rely heavily on the precise control of mechanical motion. At the heart of this motion control are electric motors, which convert electric energy into mechanical energy to drive machinery. Understanding the different types of motors, they are classified as alternating current (AC) or direct current (DC) motors, depending on the type of current that generates their magnetic field [10].

AC motors include induction motors, synchronous motors, universal motors, and switched reluctance motors. DC motors comprise brushed DC motors, brushless DC motors, stepper motors, and servomotors. We are mainly interested in the Servomotors. Before that, what do we mean by “Servo” in a Servomotor?

1.2.1 Servomechanism

Prior to the advent of electronic systems, machinery was operated manually by individuals who shifted gears, pulled levers, or applied brakes. In contemporary settings, machinery is often controlled through buttons or knobs, with electronic circuits performing many measurements and decisions automatically. In environments where electronics and mechanical systems interact, servomechanisms are commonly employed. For example, an automobile driver monitors and controls a vehicle by making continuous adjustments to the steering wheel to maintain the vehicle's trajectory. The driver's sensory input, cognitive processing, and physical actions are analogous to the functions performed by a servomechanism. However, servomechanisms execute these tasks with greater speed, sensitivity, accuracy, and consistency. [13]

1.2.2 Servo Drives

A servo drive, also referred to as a converter, receives control signals from controllers and regulate them to supply the required voltage and current to the servo motors. There are various types of servo drives, but the most common one is the torque-mode amplifier. This converts the command from the controller through PWM and inverter into the specific motor current. They are mainly used for superior positioning, speed, and motion control. Servo drives are utilized in computer numerical control (CNC) machining, factory automation, robotics, and a range of other industrial applications [7]. Figure 1.1 visually illustrates the essential functional relationship between the primary components: the motor, drive, and the control unit (located inside the case).



Figure 1.1: Lexium 32 Servo Drive and Servo motor from Schneider Electric [12]

Similar to servo motors, servo drives offer a key advantage over DC or AC motors through the integration of motor feedbacks like current and position values. This feedback enables detection and correction of deviations from the commanded motion. Servo systems also demonstrate improved operational lifespan when maintained at constant speeds compared to conventional AC wound motors. In industrial environments, both servo motors and servo drives are essential for monitoring position and controlling speed [7].

1.3 Problem Statement

In addition to standard timing and functional simulation methods for FPGA verification, functional tests can also be conducted directly on physical hardware. Traditional RTL simulations cannot fully capture physical-layer effects or interpret bitstreams, leading to undetected bugs in reconfiguration or system integration until the design is evaluated on the actual device [17]. Prior to releasing a new servo drive control board, comprehensive verification of all FPGA modules is essential to ensure that each module meets its requirements and operates reliably within the complete hardware system.

At present, verification of servo drive control boards, particularly the FPGA modules, is performed manually for each module. Although this method provides basic validation, it is resource-intensive, limited in scope, and unable to replicate the dynamic conditions encountered in actual operation. The process requires engineers to configure the bare-metal application in Vitis, connect the debugger, and complete multiple preparatory steps before testing can commence. Dependence on manual setup increases the likelihood of errors and prolongs the verification process. Furthermore, this approach is not easily scalable, as each test cycle must be configured and debugged individually. As FPGA designs become more complex, the absence of automation further impedes verification efficiency and delays product release.

The existing verification method is limited to static input testing, implemented by manually assigning fixed PWM duty cycles via AXI registers. This confines verification to steady-state behavior and does not provide insight into servo drive performance under variable conditions such as fluctuating loads, sine-wave inputs, or abrupt disturbances. In the absence of dynamic testing, comprehensive evaluation of transient responses, control-loop stability, and the system's ability to manage unexpected changes is not feasible, potentially allowing critical issues to remain undetected.

A further limitation is the requirement for physical motors and additional hardware to generate realistic feedback signals. Measuring current on the motor phase using ADCs requires dedicated hardware components, which increases costs and introduces variability due to hardware wear or changing environmental conditions. Additionally, this dependency

complicates the testing of extreme or hazardous scenarios that cannot be safely replicated with actual motors. These constraints reduce the completeness and repeatability of the verification process.

Current measurement techniques are also inefficient, as they rely on manual estimation and on converting PWM duty-cycle values to current levels using motor parameters. These procedures are time-consuming and must be repeated for each test cycle. The absence of automated digital tools, such as FFT-based analysis, prevents improvements in speed, accuracy, and error detection.

An additional challenge of the present approach is the inability to test FPGA modules independently. For instance, verifying the current-sense module requires the PWM module to be operational in order to drive the motor and generate measurable current. This interdependence prevents isolated verification of individual modules and may obscure integration issues until late in the development process. The absence of modular testing reduces the effectiveness of incremental verification, increases risk, and can result in costly redesigns or field failures. Collectively, these challenges underscore the necessity for a more automated, flexible, and comprehensive FPGA verification methodology for servo-drive control boards.

1.4 Desired Outcome

The servo drive control board under consideration (Figure 1.2) is based on the Multi-Processor System-on-Chip, which integrates application processors alongside with real-time processors. The system incorporates various hardware modules to support diverse functionalities, including motor control, signal filtering, communication interfaces, diagnostics, and debugging.

The primary objective of this project is to develop a robust proof-of-concept for a dedicated FPGA Verification Platform. This platform will enable comprehensive verification and testing of FPGA modules for the control board by emulating required inputs and triggers and verifying module outputs against expected behavior. As a result, the platform will

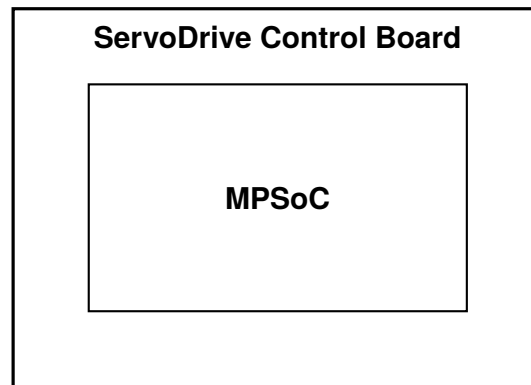


Figure 1.2: Control Board Component Diagram

substantially enhance test coverage compared to the simulation-based methodologies. This approach directly addresses the limitations identified in the previous section.

The proposed platform will automate the verification workflow, reducing or eliminating manual intervention and significantly accelerating the test cycle. It will support both static and dynamic test scenarios, enabling more exhaustive functional assessment under conditions representative of real-world operation. A key benefit is the ability to test individual modules independently of other FPGA components or external hardware, which simplifies debugging and improves the reliability of module-level verification. The system will incorporate mechanisms for processing complex data and delivering clear, actionable results, thereby eliminating reliance on slow, error-prone manual analysis. Additionally, the platform will be designed for scalability, ensuring compatibility with various FPGA versions of the control board and accommodating new test cases as design requirements evolve. By integrating these capabilities into a structured, intuitive workflow, the platform will deliver a significantly improved user experience compared to existing verification practices and a more efficient, reliable, and future-oriented verification methodology.

2 Technical background

This chapter presents the technical background necessary to understand the design and implementation of the FPGA verification platform. It begins with an examination of the Zynq UltraScale+ MPSoC architecture, which serves as the primary technology for the servo drive control board. Subsequent sections address key implementation topics, including clocking strategies and clock-domain crossing management. The chapter concludes with an overview of various FPGA verification methods, emphasizing in-system testing and the AMBA AXI protocols that are essential for the platform's communication infrastructure.

2.1 Zynq UltraScale+ MPSoC FPGA

Organizations developing advanced intelligent systems that leverage both hardware and software programmability can accelerate product development, increase value, and continuously enhance their products through flexibility, reuse, and rapid upgrades. In response to these evolving requirements, Xilinx introduced the UltraScale MPSoC architecture. This platform builds on the earlier 28-nanometer Zynq-7000 All Programmable SoC and UltraScale FPGA families to address the higher performance and functionality demanded by next-generation applications [18].

Essential features of Zynq UltraScale+ MPSoC include [18]:

1. **Optimized Processing Engines for Specific Tasks:** This architecture employs both hardware and software processing engines to address bottlenecks in digital signal processing, graphics, networking, real-time control, and general computing. The heterogeneous processor mix enhances efficiency in terms of performance, power consumption, and cost.
2. **Scalability to 64-Bit Processing:** Processing capabilities scale from 32-bit to 64-bit and fully support virtualization. The system interconnect, peripherals, and memory architecture provide a large address space, enabling future scalability.

3. **Advanced Interconnect and Memory Architecture:** A high-speed interconnect and advanced memory system enable all processing engines to share data efficiently, minimizing latency and maintaining optimal system performance.
4. **FPGA Scalability and ASIC-Class Performance:** Programmable logic with ASIC-like features enables hardware acceleration for demanding tasks. The Vivado design tools facilitate real-time performance and rapid hardware customization.
5. **Multi-Level Security, Safety, and Reliability:** The architecture incorporates robust anti-tamper protections, trust features, and reliable operation, making it suitable for Internet of Things and mission-critical applications in challenging environments.
6. **Advanced Power Management:** Granular power control across processors enables the system to adjust power consumption dynamically, reducing both idle and active power according to workload demands.
7. **16nm FinFET Performance per Watt Advantage:** Utilizing TSMC 16nm FinFET technology, UltraScale MPSoCs deliver up to 60% improved performance per watt compared to 28nm devices, while mitigating the risks associated with custom ASIC development.
8. **High-Level Design Abstractions:** Support for C/C++, SystemC, OpenCL, MATLAB, and other tools makes it easy to speed up software-to-hardware tasks and reuse IP-based subsystems, which boosts productivity.

The integration of these elements results in a cohesive architecture that defines the UltraScale MPSoC design. In summary, the Zynq UltraScale+ MPSoC architecture integrates diverse processing elements, scalable interconnects, and advanced design tools to provide high performance and flexibility. However, the system's efficiency and reliability depend on precise clock management and synchronization across processing domains. The following section examines the challenges of managing multiple clocks in FPGA design and discusses techniques that maintain system stability and predictability.

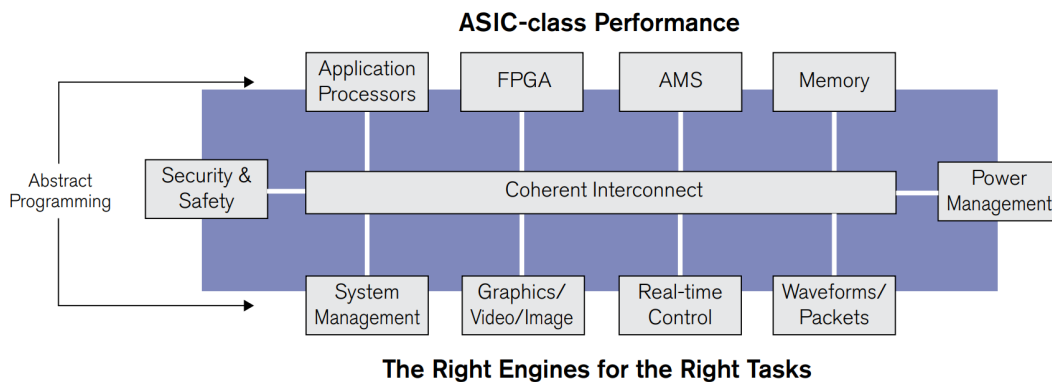


Figure 2.1: The Xilinx UltraScale MPSoC architecture [18]

2.2 Clock Nuances in FPGA-Design

Reliable digital operation in modern FPGA-based systems requires effective clocking strategies. As system architectures integrate additional processing elements and peripherals, the complexity of managing multiple clock domains increases. The techniques employed to generate, distribute, and synchronize clocks significantly influence system performance, timing accuracy, and overall reliability. This overview examines two fundamental aspects of FPGA clock design: the use of the Xilinx Clocking Wizard for clock generation and the management of clock-domain crossings using CDC IP cores. Both elements are crucial to achieving stable, predictable system operation.

In FPGA architectures, clock signals define the timing for sequential logic, coordinating the operation of flip-flops, memory interfaces, and high-speed data paths. While simple designs may utilize a single global clock, more complex systems typically require multiple clock domains to accommodate functional blocks operating at varying frequencies. Although this strategy enhances performance and flexibility, it introduces substantial challenges in clock management and synchronization.

The Xilinx offers Clocking Wizard IP (Figure 2.2) as a flexible tool for clock generation and management. It streamlines the creation of HDL wrappers for clock circuits tailored to specific requirements. The wizard helps configure attributes for clocking primitives and lets users override automatically determined parameters. It also generates a summary of

timing parameters from Xilinx timing analysis tools. The Clocking Wizard facilitates the creation of clocking circuits configured for specific output clock frequencies, phases, and duty cycles by utilizing Mixed-Mode Clock Manager (MMCM) or Phase-Locked Loop (PLL) primitives [21].

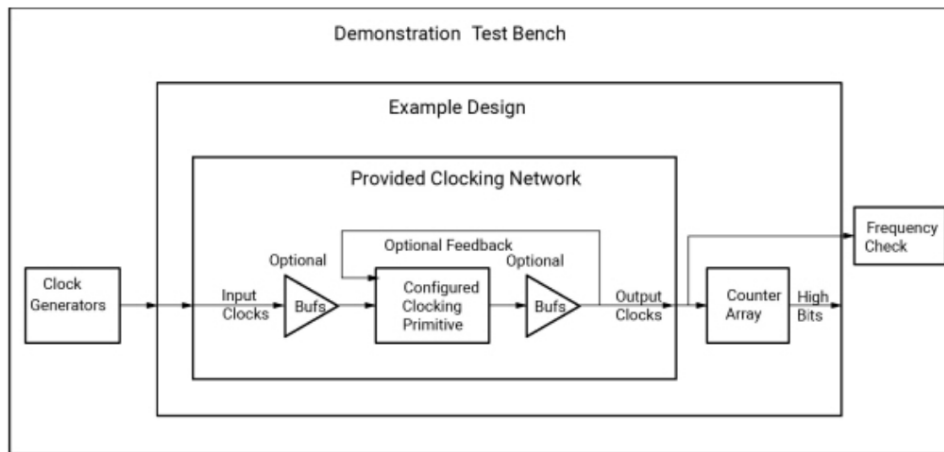


Figure 2.2: Clocking Wizard Block Diagram [21]

While the Clocking Wizard simplifies clock creation, the presence of multiple independent clocks introduces a new challenge: clock domain crossing (CDC). CDC problems happen when signals that are not synchronized move between different clock domains. This can cause metastability, leading the design to behave unpredictably. In some cases, metastability can cause excessive current and even damage the chip. Because the delay for signals crossing between clock domains cannot be guaranteed, this unpredictability is a concern [16]. To prevent these issues, it is important to thoroughly test designs with multiple clock domains and ensure CDC-related problems do not affect system operation.

One way to mitigate the CDC when two different clock frequencies are involved in an FPGA Design is to use Xilinx CDC IP core. An example implementation of this approach is illustrated in the accompanying figure 2.3. The Clocking Wizard module (clk_wiz.1) generates a 192 MHz clock signal derived from the 100 MHz system frequency. This 192 MHz output drives the debugger IP for on-chip signal monitoring and debugging.

The PWM generator module generates PWM signals at 100 MHz, the system clock frequency. Because the debugger module operates at 192 MHz, direct capture of the PWM signals is not possible due to the differing clock domains. To synchronize these signals,

the Xilinx CDC IP Core (`xpm_cdc_gen_3`) is inserted between the PWM generator module and the debugger.

The CDC IP core receives the source clock (100 MHz), the destination clock (192 MHz), and the source signal (PWM output). It generates a synchronized version of the input signal, aligned with the destination clock domain. This synchronized signal is routed to the debugger, enabling reliable signal capture without metastability.

This configuration ensures that signals transferred from the PWM generator module at 100 MHz are safely synchronized and observable in the 192 MHz domain used by the debugger module. As a result, the design maintains deterministic and stable behavior across multiple clock domains.

Effective clock management is essential in designs with multiple clock domains to ensure correct operation and address specific verification challenges. Signal transfers between asynchronous domains, even when using CDC IP cores for synchronization, can result in subtle timing and functional issues that are difficult to detect. As FPGA designs become more complex, with higher speeds and diverse processing elements, comprehensive verification is required to maintain reliable, predictable performance. The following section examines several FPGA verification methods, including simulation, in-system testing, and hardware-in-the-loop approaches, which facilitate the identification and resolution of timing and functional issues in modern FPGA designs.

2.3 Types of FPGA Verification

Verifying FPGA designs involves specific tasks and tools tailored to the technology. Unlike ASIC designs, FPGA implementations typically lack extensive verification resources and dedicated planning. Often, verification is treated as secondary because FPGAs can be reprogrammed. Neglecting comprehensive verification planning may result in substantial development delays and disruptions, which are comparable to the setbacks caused by unexpected technology re-spins. Assuming verification is low-risk because errors can be corrected by reprogramming overlooks the effort required to identify and resolve issues, especially in complex designs featuring multiple clock domains. Delays in

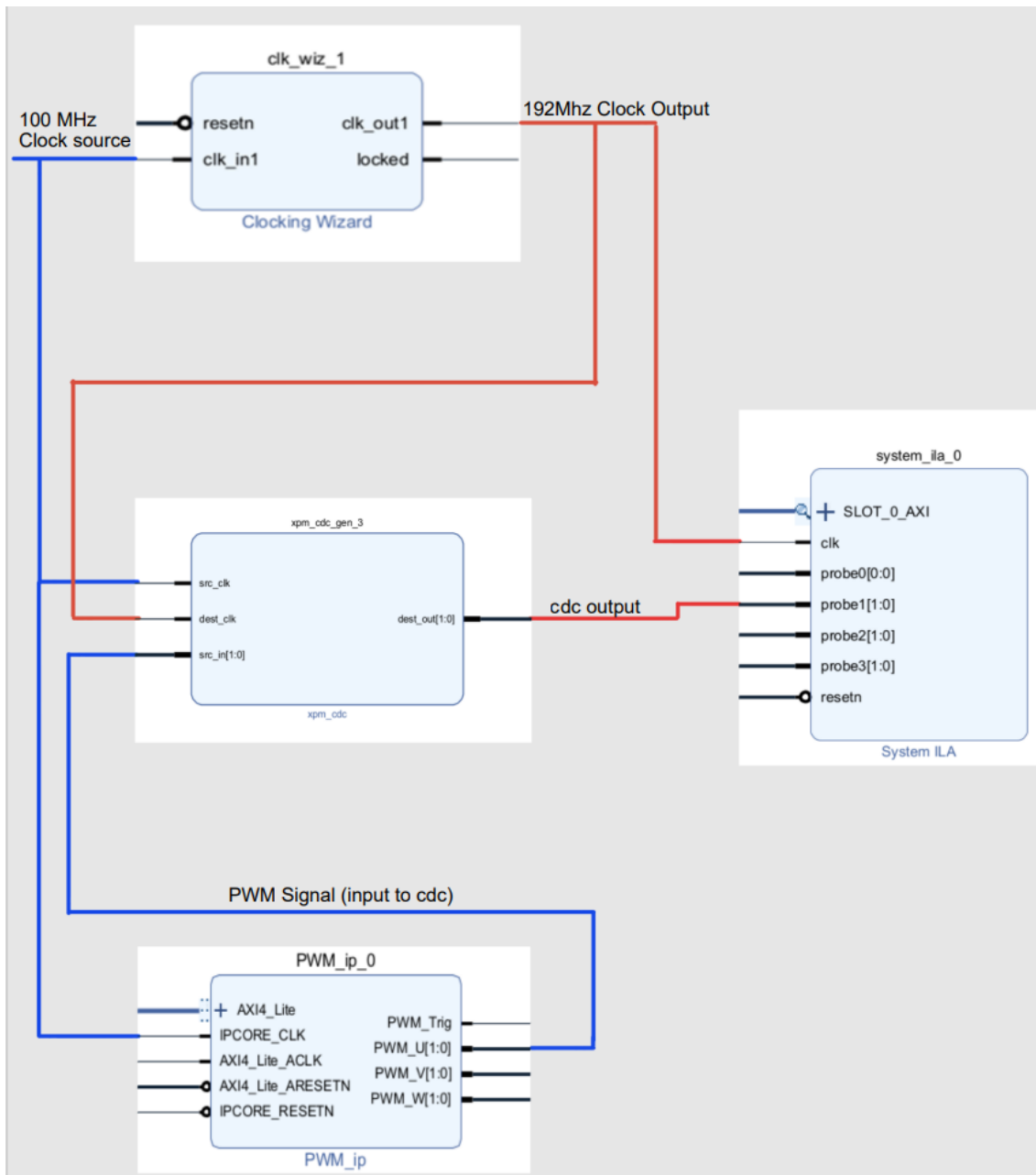


Figure 2.3: An example illustrating the use of Clocking Wizard and Xilinx CDC IP Core

hardware verification can also affect other development areas, such as software and system integration, due to their interdependence [6].

A structured hierarchy of verification methodologies helps reduce FPGA-related risks. Initial verification focuses on identifying major functional issues at a high level. As the process advances toward a fully operational, high-speed design, errors become more subtle and are often related to timing or specific data conditions. Some issues are so rare they may only appear after extended operation. In these cases, the test environment must capture enough information for thorough analysis and resolution [6].

2.3.1 Simulation Verification

In FPGA functional verification, simulation plays a vital role in the design process. It enables engineers to model and evaluate digital circuits prior to hardware implementation, helping to detect design flaws and confirm correct operation. Through simulation, mathematical representations of FPGA systems are created, various test scenarios are applied, and the resulting outputs are analyzed.

This approach provides valuable insights into system behavior, assists in performance evaluation, and supports validation of design choices. A solid understanding of simulation principles allows designers to optimize FPGA architectures, minimize development risks, and enhance the overall reliability and robustness of digital systems [8].

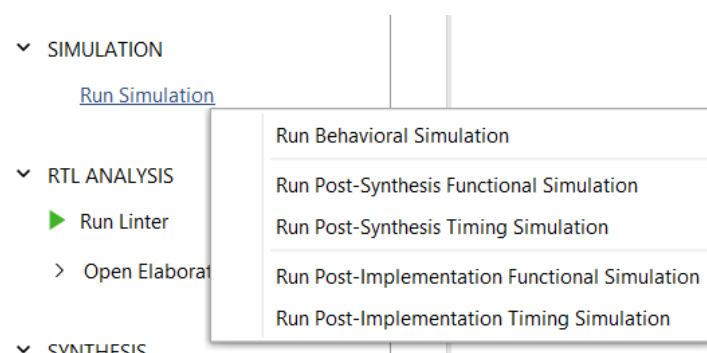


Figure 2.4: Types of FPGA Simulation Verification

The figure above illustrates the various existing simulation verification methods.

- **Behavioral / RTL Simulation:** RTL-level simulation enables simulation and verification of a design before translation by synthesis or implementation tools. Designs can be verified at the module, entity, block, device, or system level.

RTL simulation is typically conducted to verify code syntax and confirm correct functionality. At this stage, the design is described in RTL, and timing information is not required [1].

There are two main approaches to behavioral or RTL simulation. The first uses event-driven execution, where the simulator processes generated code based on events. The second is a compiled simulation, which executes the code directly at specific time points. Compiled simulation supports only zero-delay scenarios and assumes the circuit is synchronous. In both methods, statements at lower levels are executed sequentially, while at the module level, execution is event-driven. For example, statements within a VHDL process block are executed sequentially [24].

- **Post Synthesis Simulation:** A synthesized netlist is generally simulated to verify that the design fulfills functional requirements and exhibits the intended behavior. Timing simulation using estimated timing values may also be conducted at this stage, although this practice is less common.

The functional simulation netlist is organized hierarchically and folded, with expansion down to the level of primitive modules and entities[1].

- **Post Implementation Simulation:** After implementation, either functional or timing simulation may be performed. Timing simulation most accurately replicates the process of downloading a design to a device. This method verifies that the implemented design meets both functional and timing requirements and demonstrates the expected behavior within the device[1].

2.3.2 In-System Verification

Embedded logic analyzers provide the capabilities of traditional external logic analyzers but are implemented within the FPGA fabric using dedicated IP cores that connect to specific internal signals. These analyzers capture signal activity according to user-defined trigger conditions and store the resulting data in on-chip memory for subsequent analysis. Although some runtime configuration is possible, most trigger conditions and signal selections are established during the design phase. Debug cores require FPGA resources, particularly block RAMs for trace storage, and modifications to the instrumentation typically necessitate re-synthesizing the design. Nevertheless, embedded logic analyzers offer near real-time signal visibility and can function at or near the target system frequency.



Figure 2.5: System ILA IP Core

Earlier toolchains featured Xilinx ChipScope Pro and Intel (formerly Altera) SignalTap II as primary examples of embedded logic analyzer solutions. In current workflows, ChipScope Pro has been replaced by the Vivado Integrated Logic Analyzer and System ILA IP (Figure 2.5) from AMD (Xilinx), while SignalTap II continues as Intel's integrated solution for Quartus-based designs. Additional tools, including the Synopsys Identify RTL Debugger, offer advanced source-level instrumentation and debugging capabilities for FPGA development.

2.3.3 Software-Driven Verification

Software-driven verification is an advanced methodology for FPGA validation that utilizes embedded processors within the FPGA, such as ARM cores in Xilinx Zynq, MicroBlaze soft processors, or Intel Nios II, to validate hardware modules using executable test software. In this methodology, the processing system executes test routines or firmware that interact with the programmable logic via standard on-chip communication interfaces such as AXI4-Lite (Figure 2.6) or AXI4-Stream. This approach enables real-time verification of design functionality under realistic operating conditions and serves as a bridge between pre-silicon verification and post-silicon validation [5]. The PS configures, stimulates, and monitors PL blocks by reading and writing AXI-mapped control and status registers, while embedded test software automates both functional and performance verification sequences.

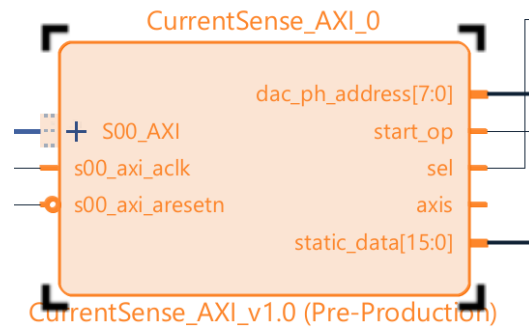


Figure 2.6: An example for AXI4 IP Core

Software-driven verification is increasingly adopted in modern SoC-FPGAs due to its flexibility and scalability. This methodology accelerates verification and enhances design observability and performance monitoring IP cores. As shown in recent IEEE research, frameworks such as HIVE [5] demonstrate that running test software on embedded processors significantly improve coverage for hardware and software interaction, AXI protocol compliance, and timing validation across the PS–PL boundary by running test software on embedded processors

2.3.4 Hardware-in-loop Verification

Traditional software-based simulation is limited in its ability to replicate operational conditions. Hardware-in-the-loop (HIL) simulation mitigates this limitation by integrating real hardware components into the simulation environment. In real-time HIL simulation, physical hardware replaces emulated hardware or control logic within the simulation model and interacts directly with computational models. This method increases simulation fidelity, provides access to hardware-specific features not present in software-only models, and decreases the likelihood of undetected errors during final field testing [26].

A special subset of HIL, known as **FPGA-in-the-Loop (FIL)** verification, focuses on integrating FPGA-based designs as the device under test (DUT) within the simulation environment. The DUT is implemented on a physical FPGA, while a separate hardware platform, such as another FPGA or a real-time controller, supplies input signals and captures output responses for analysis. This setup enables real-time verification of FPGA logic for both functional correctness and timing accuracy. Unlike full HIL, FIL does not simulate physical system dynamics but instead offers a high-fidelity environment for testing FPGA modules with precise synchronization and low latency, which is especially valuable for high-speed digital systems.

A review of FPGA verification methods demonstrates that real-time observation of internal signals is critical for debugging and validating complex designs. Although simulation and software-based verification are valuable, specific timing behaviors, particularly in systems with multiple clock domains, are observable only on physical hardware. The ILA IP Core fulfills this requirement by offering on-chip tools for real-time signal monitoring. These tools enable designers to detect and resolve subtle functional and timing issues. The following section describes the ILA's types, features and practical applications in FPGA design and verification.

2.4 System Integrated Logic Analyzer

The Integrated Logic Analyzer (ILA) is a built-in debug IP core provided by Xilinx that allows in-system probing of internal FPGA signals. Rather than routing signals out to

external pins, the ILA captures internal signal waveforms in on-chip memory (e.g. BRAM / UltraRAM) and then makes them available for visualization.

The System ILA IP is a recent solution intended for block designs and systems that use an IP integrator or SoC architecture. This tool enables the probing of protocols such as AXI4 and AXI Stream, and provides enhanced integration with Vivado debug tools.

In FPGA design, signals and interfaces are connected to the System ILA using probe and slot inputs, as illustrated in figure. These signals and interfaces are sampled at the design's operating frequency and stored in on-chip Block RAM (BRAM) [20].

Communication with the System ILA core is established through an automatically generated debug core hub that connects to the FPGA JTAG port. After programming the design into the FPGA, the Vivado Hardware Manager software enables configuration of trigger events for System ILA measurements. When the trigger condition is met, the sample buffer collects data and transmits it to the Hardware Manager for visualization and analysis in the waveform window [20].

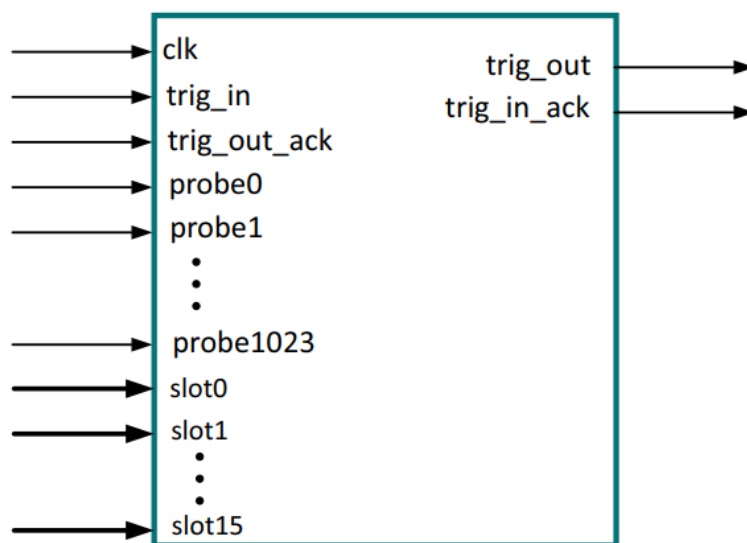


Figure 2.7: System ILA Symbol [20]

Standard FPGA logic manages probe sampling and trigger operations, while on-chip BRAM temporarily stores captured data until the software retrieves it. The process,

encompassing triggering, data capture, and communication with the System ILA core, operates automatically without requiring user intervention [20].

The System ILA can monitor interface-level signals, providing transaction-level insights, such as tracking outstanding transactions on AXI4 interfaces. This feature establishes the System ILA as a valuable tool for analyzing AXI-based communication in FPGA and SoC designs. The System ILA captures and displays AXI4 transactions in real time, enabling verification of protocol compliance, observation of data flow between master and slave components, and identification of potential bottlenecks or design issues. Effective use of the System ILA for monitoring and debugging requires a thorough understanding of AMBA AXI interfaces.

The figure 2.8 presents a practical example of the System ILA capturing AXI4 write transactions. The waveform displays key AXI signals, including **AWVALID**, **AWREADY**, and **AWADDR**, demonstrating the ILA's capability to provide real-time visibility into the transaction process and to support verification of correct protocol operation.

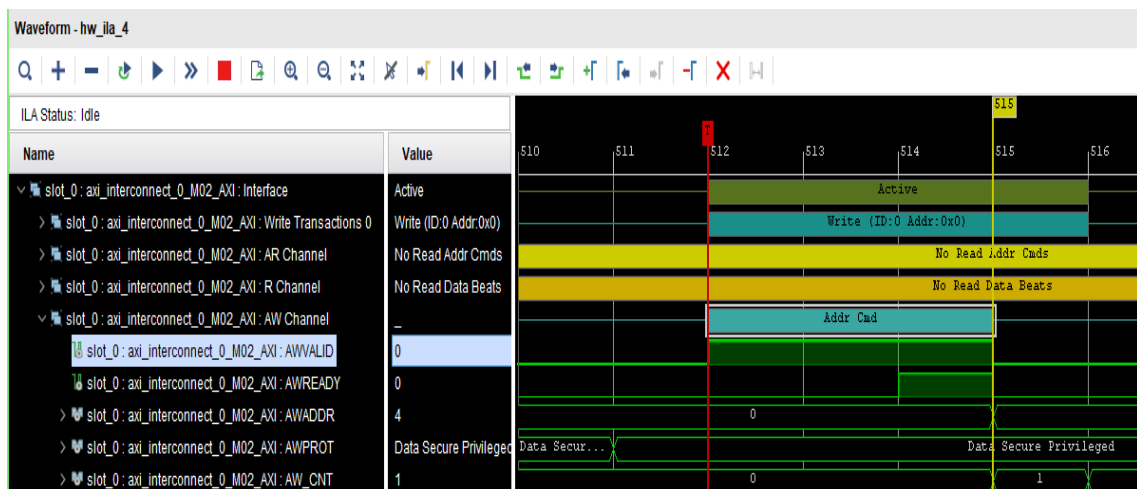


Figure 2.8: System ILA capture of AXI4 write transaction signals.

2.5 AMBA AXI Interfaces

The ARM Advanced Microcontroller Bus Architecture (AMBA) is an open standard specification for on-chip communication and interconnection that manages functional

blocks within a SoC design. The AMBA protocols define the mechanisms that enable communication between these functional blocks [3].

The Advanced eXtensible Interface (AXI) constitutes the third generation of the AMBA standard, introduced in the AMBA 3 specification. AXI is designed for systems that demand high performance and rapid clock speeds, incorporating features that enable efficient, high-speed interconnects in sub-micrometer technologies. In 2010, ARM introduced the AMBA 4 specifications, beginning with AXI4, and subsequently released the AXI Coherency Extensions (ACE) in 2011 to improve system coherency. The AXI4-Stream protocol enables unidirectional data transmission from the manager to the subordinate. Its simplified signal routing makes it especially suitable for implementation in FPGA-based systems [3].

The AXI protocol specifies the processes for reading, writing, caching data, and translating addresses. These operations utilize several distinct communication channels [4]:

- Write request signals are denoted by the prefix **AW**.
- Write data signals are denoted by the prefix **W**.
- Write response signals are denoted by the prefix **B**.
- Read request signals are denoted by the prefix **AR**.
- Read data signals are denoted by the prefix **R**.

A request channel transmits control information specifying the type of data to be transferred. Data is exchanged between the Manager and Subordinate through the following channels [4]:

- **Write data channel:** The Manager transmits data to the Subordinate via this channel. Upon completion, the Subordinate notifies the Manager through the write response channel.
- **Read data channel:** The Subordinate transmits data to the Manager through this channel.

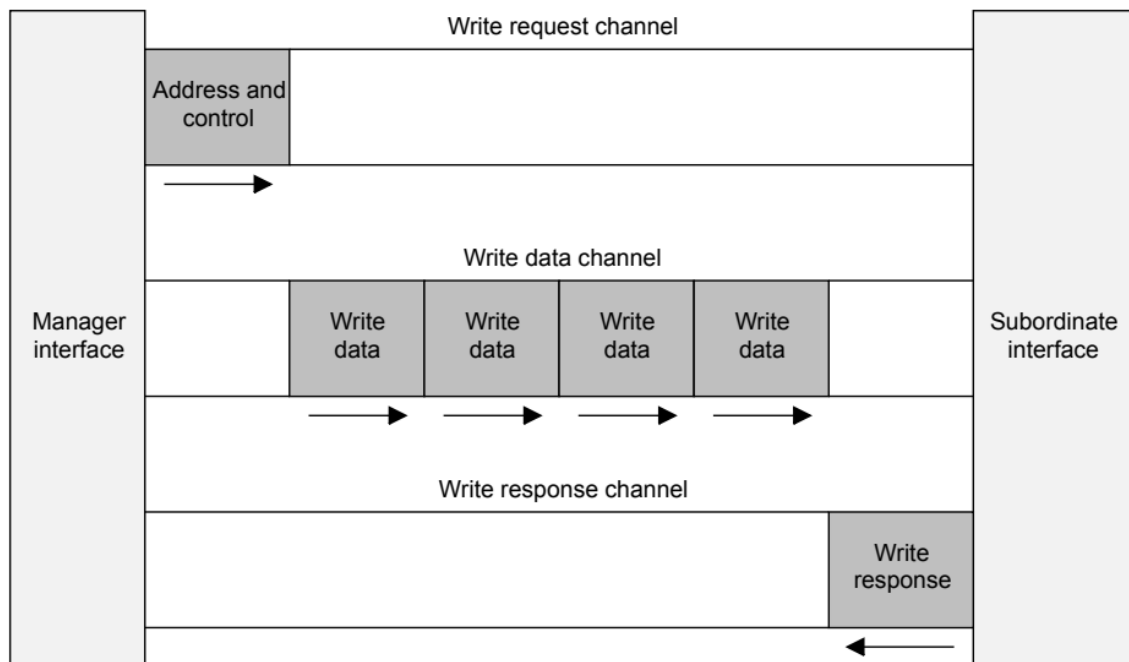


Figure 2.9: The figure illustrates the operation of a write transaction, which utilizes the write request, write data, and write response channels [4].

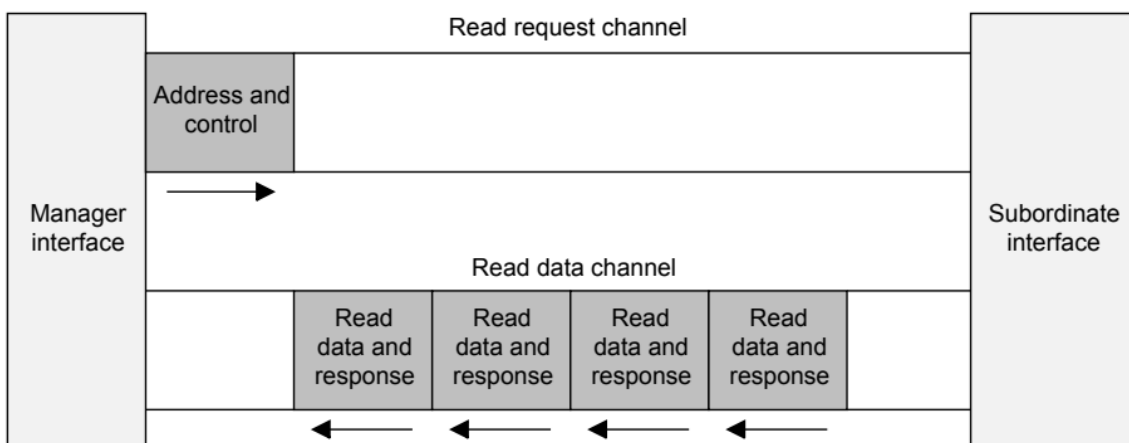


Figure 2.10: The figure depicts a read transaction, demonstrating the use of the read request and read data channels [4].

Write and Read Request Channels

The AXI protocol specifies separate channels for write and read requests. Each channel transmits the address and control information required to initiate a transaction.

Write Data Channel

The write data channel transfers data from the Manager to the Subordinate, supporting data widths up to 1024 bits. A byte-lane strobe signal indicates valid bytes for every eight bits. Data on this channel is always buffered, allowing the Manager to initiate multiple write transactions consecutively without waiting for confirmation from the Subordinate.

Write Response Channel

The Subordinate utilizes the write response channel to indicate completion of a write transaction. Each transaction requires a completion signal on this channel. As illustrated in Figure 2.9, the response is provided only after the entire transaction is complete, rather than after each data segment.

Read Data Channel

The read data channel transmits both read data and response information from the Subordinate to the Manager. Similar to the write data channel, it supports data widths up to 1024 bits.

AXI Interface Variants

The AMBA AXI comprises three primary types. Each variant provides a distinct balance between complexity and performance to address diverse design requirements [19].

- AXI4 (Full) is a high-speed, memory-mapped interface that supports burst transfers of up to 256 beats, enables concurrent transactions, and manages out-of-order responses. These features make it suitable for memory-intensive applications.
- AXI4-Lite is a simplified version of AXI4 that supports basic single-read or single-write operations. It is typically used for control and status registers, where resource efficiency is prioritized over speed.

- AXI4-Stream is a unidirectional, high-speed streaming interface that omits the address phase and utilizes only data and handshake signals. It is commonly implemented in real-time data pipelines such as video processing, digital signal processing, and communication systems.

Collectively, these three interface types establish a flexible interconnect system. AXI4 connects to main memory, AXI4-Lite handles configuration and control, and AXI4-Stream facilitates continuous data flow. This architecture ensures consistent integration across intellectual property cores in Vivado-based system-on-chip designs[4][19].

The technical elements presented in this chapter constitute the primary components of the proposed verification platform. With the theoretical foundation established, the subsequent chapter will detail the concepts and models used to design the platform's architecture and demonstrate how these technologies contribute to a robust, automated solution for PL module verification.

3 Concept and Modeling

This chapter presents the core concepts, architectural models, and hardware components selected for the FPGA Verification Platform Proof of Concept (PoC). The discussion begins with an examination of the Servo Drive Control Board Architecture, which defines the operational context and identifies the Design Under Test (DUT) modules. Subsequently, a Requirement Analysis is conducted to justify selecting the Enclustra Mercury XU5 and the DAC80508 as the primary verification hardware. The Block Design of the proposed PoC is then introduced, illustrating the integration of these components into a closed-loop system for automated FPGA module verification. The chapter concludes with a review of related work and case studies in FPGA and Hardware-in-the-Loop (HIL) Verification, situating the platform's contribution within the broader research landscape.

3.1 Servo Drive Control Board Architecture

Modern servo drives integrate advanced control electronics, optimized power efficiency, and comprehensive safety mechanisms within a compact, modular architecture. In industrial automation applications, such as those utilizing Schneider Electric's PacDrive 3 platform, servo drives serve as the central components for synchronizing multi-axis control systems.

The PacDrive 3 platform (figure 3.1) comprises a central Logic Motion Controller (LMC) and multiple Lexium 62 servo drives, which are interconnected via a Sercos III real-time Ethernet network. The LMC manages motion planning, generates movement trajectories, and synchronizes all servo axes. Each Lexium 62 drive utilizes integrated electronics to regulate current and speed, continuously transmitting feedback to the LMC. This configuration facilitates system scalability, minimizes wiring requirements, and ensures precise real-time coordination among motion axes. Each servo drive has the following primary features [11] [29] :

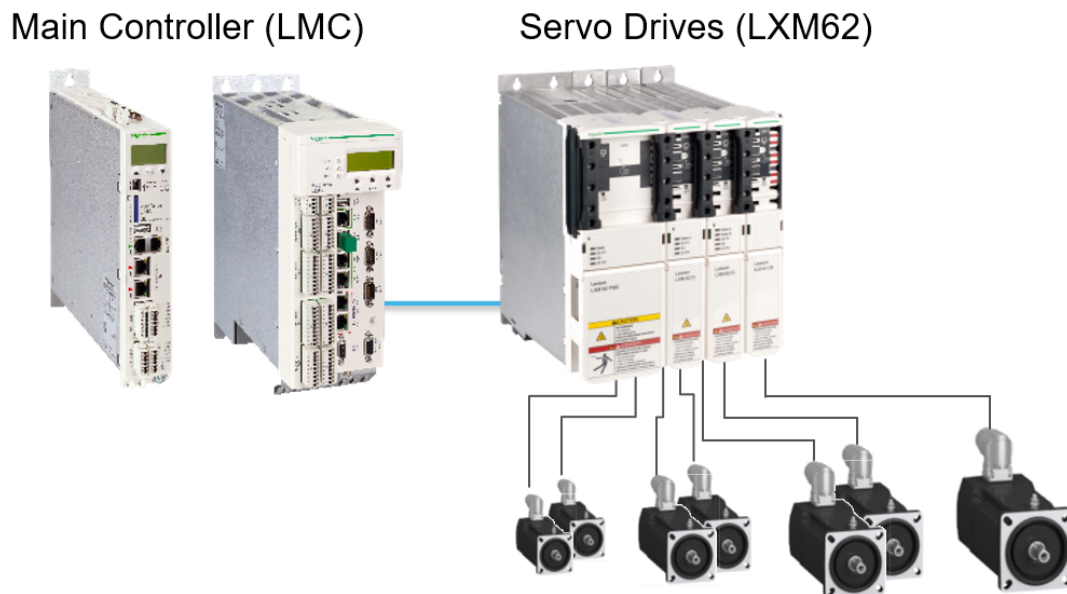


Figure 3.1: PacDrive 3 Platform. ServoDrive - System Architecture

1. **Motion Control and Application Capabilities:** Lexium 62 drives employ advanced motion control and feedback systems to achieve rapid, smooth movements with minimal contouring error. This enables precise positioning and consistent performance in demanding servo applications. The drives are compatible with Schneider Electric's Lexium SH3, MH3, and SHS stainless-steel servo motors, and can also interface with select third-party linear, torque, or asynchronous motors via encoder adapters, provided these motors comply with over voltage category III standards
2. **Fast Connection Technology:** A key advantage of the Lexium 62 series is its Fast Connection design, which streamlines installation and integration. Each drive module features front-facing plug-in connections for the DC bus, 24 V power, and Sercos III communication, eliminating the need for complex back plane wiring. Through the Sercos motion bus, single-and double-axis drives integrate into the control network and communicate directly with the central Logic Motion Controller (LMC). The shared DC bus supplies up to 120 A of continuous current, enhancing energy efficiency and simplifying cabinet wiring for multi-axis systems.

3. **Safety and Reliability Features:** Safety is integrated into both the hardware and communication systems. Standard drives incorporate a hardwired Safe Torque Off (STO) feature, certified to SIL 3/PLe, ensuring the motor remains torque-free during emergencies or faults. Advanced models provide additional safety functions beyond Sercos, including Safe Stop, Safe Operating Stop, and Safe Limited Speed, in compliance with IEC 61508 and EN ISO 13849-1. In multi-axis systems, Lexium 62 modules utilize a shared power supply and DC link, reducing cabinet space requirements and costs while maintaining safety.
4. **Internal Functionalities and Drive Logic:** Beyond motion control, the drives incorporate multiple protective and adaptive features. These include torque limits that adjust with acceleration to prevent mechanical stress, detection of mechanical overloads, brake-release verification, and velocity control without an encoder when paired with BMP motors. These intelligent features enhance reliability, safeguard components, and enable flexible operation even in the absence of complete encoder feedback.

Figure 3.2 illustrates the internal structure of a typical servo drive. The diagram distinguishes the Control Electronics, responsible for low-voltage and cold-side control, from the Power Electronics, which manage high-voltage and hot-side control.

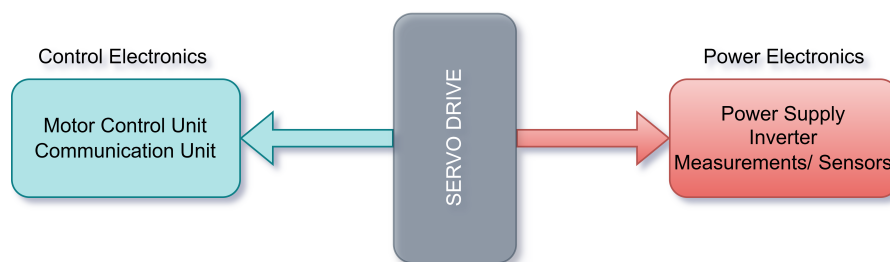


Figure 3.2: Architectural division of Servo Drive Control Board

1. Hot Side (Power / Motor Zone):

- This section comprises the primary high-voltage and high-current components, including inverter bridges, DC bus elements, braking circuits, and motor windings.

- This delivers the necessary currents and voltages to the servo motor, facilitating precise regulation of torque, speed, and position.

2. Cold Side (Control / Logic Zone):

- This section contains low-voltage electronic components, including controllers, isolated gate drivers, sensors, feedback circuits, logic circuits, and communication interfaces.
- The control logic and input/output circuits typically operate at supply voltages of 24 V or lower.

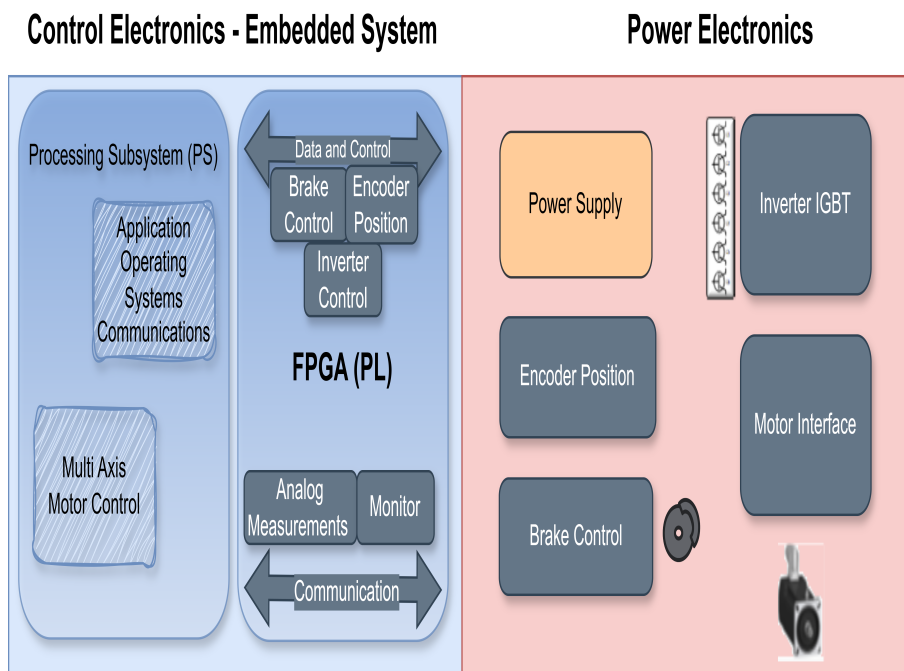


Figure 3.3: Internal architecture of the servo drive showing separation between control (cold side) and power (hot side) electronics

In multi-axis servo drive systems, all drives utilize a shared high-voltage DC bus as a central energy source. In contrast, single-axis configurations employ a dedicated power supply module that converts AC mains into a stable, high-voltage DC link and provides 24 V to the control electronics. Large capacitors on the DC bus store energy, minimize voltage ripple, and absorb excess energy during braking or deceleration.

The inverter converts DC power into controlled three-phase AC using six insulated-gate bipolar transistors (IGBTs) arranged in a three-phase bridge configuration. Each IGBT is switched on and off by high-frequency PWM signals generated by the FPGA or controller. These signals regulate the voltage supplied to each motor phase, producing current waveforms that closely approximate a sine wave with the desired amplitude, frequency, and phase. Unlike standard AC mains, which cannot be dynamically adjusted, this controlled AC enables variable speed operation, precise torque control, and comprehensive closed-loop regulation of motor position and speed. Continuous adjustment of the PWM duty cycle based on sensor feedback ensures smooth motor operation, supports regenerative braking, and enables four-quadrant operation, thereby meeting the stringent requirements of modern industrial servo systems.

An isolation barrier separates the high-voltage (DC bus components) and logic zones (24V low voltage components) to prevent high voltage from reaching the logic side and to ensure galvanic separation. This barrier safeguards low-voltage logic circuits and enhances overall system safety. The cold-side control electronics, illustrated in figure 3.3, and other low-voltage components typically incorporate a Zynq UltraScale+ MPSoC FPGA. This FPGA is responsible for logic operations, computation, and real-time control. The PS executes high-level functions, including motor and current control loops, communication, and diagnostics. The PL performs time-critical operations, including brake control, inverter control, encoder position tracking, and safety monitoring. The FPGA and processor utilize shared data interfaces to exchange control parameters and feedback in real time.

Within this architecture, the FPGA-based control electronics form the core of the cold-side logic domain. These FPGA modules represent the DUT examined in this thesis. The following section provides a detailed description of their operation, including their inputs and outputs.

3.2 Understanding the DUT

The Figure 3.4 presents the block-level design of the MPSoC FPGA implemented on the control board. The ServoDrive system utilizes the Zynq UltraScale+ CG variant,

which integrates a dual-core Arm Cortex-A53 application processor and a dual-core R5F real-time processor. In this diagram, blue blocks indicate FPGA modules implemented and requiring comprehensive testing prior to the release of the control board. These modules perform essential functions to maintain the reliable operation of the ServoDrive system. Rigorous testing is necessary to ensure system stability, accurate signal processing, and compliance with real-time performance requirements.

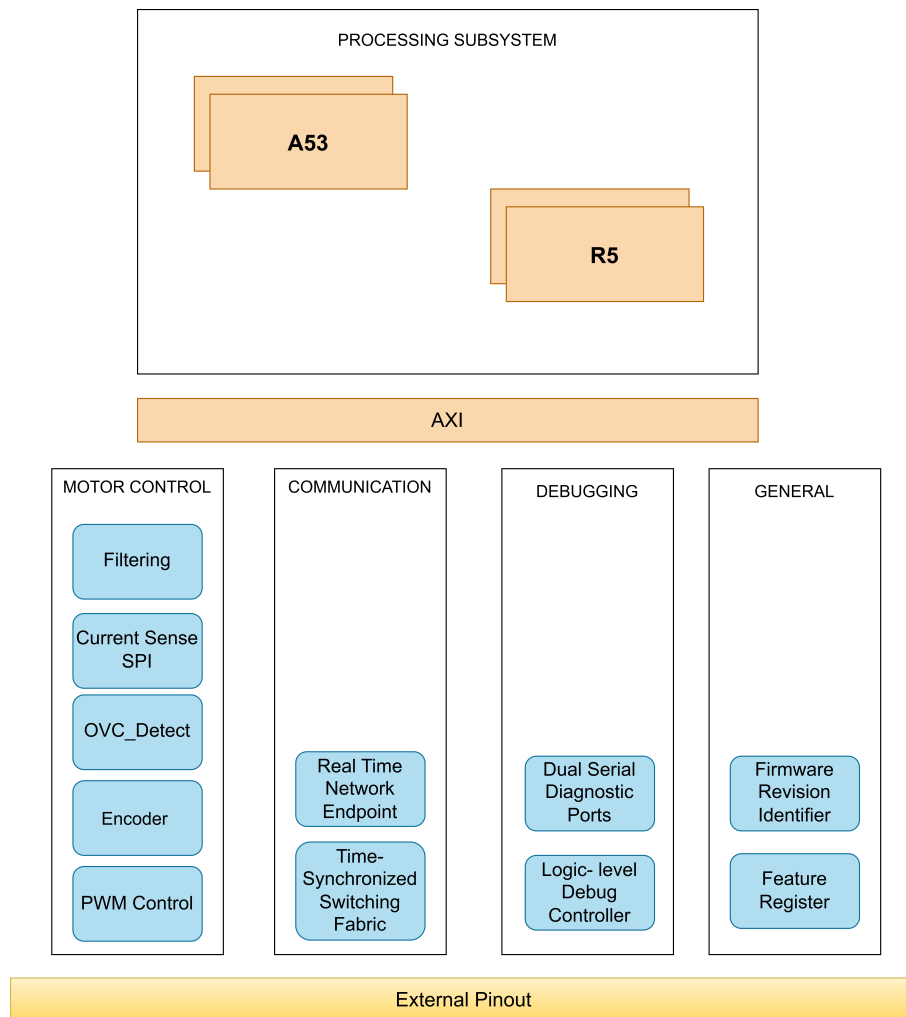


Figure 3.4: Control Board MPSoC View

This thesis examines FPGA hardware modules designed for motor control and protection. In particular, it analyses the Current Sense SPI module, which interfaces with the ADC to acquire motor feedback (serving as the primary input to the servo drive’s current control algorithm) and to monitor currents. The OVC_Detect module, which identifies over-current

conditions, and the PWM Control module, which generates pulse-width modulation signals for motor drive, are also evaluated. Focusing on these modules enables a comprehensive assessment of the system's critical control and safety features, as well as a systematic evaluation of the FPGA implementation's overall functionality and performance.

3.2.1 CurrentSense SPI Path

The figure 3.5 depicts the Current Sense path on the control board. The board's analog-to-digital converters (ADCs) continuously acquire feedback signals from the motor phases. These analog signals, representing the instantaneous motor current, are converted to digital values and transmitted to the Current Sense SPI module. The module processes these values and generates 12-bit digital outputs for subsequent control and monitoring. This signal path serves as the DUT in this thesis and functions as a critical interface between the motor hardware and the digital control logic.

The primary objective of this work is to generate feedback signals that would typically originate from the motor hardware. This method enables testing and verification of the Current Sense SPI module in a controlled environment. By producing both static and dynamic test signals, the module's response to various conditions, such as constant currents, abrupt changes, and variations in motor load, can be thoroughly evaluated. This methodology enables a comprehensive assessment of the accuracy, resolution, and timing of the Current Sense SPI outputs, ensuring the module meets all specified requirements before integration with the actual motor system.

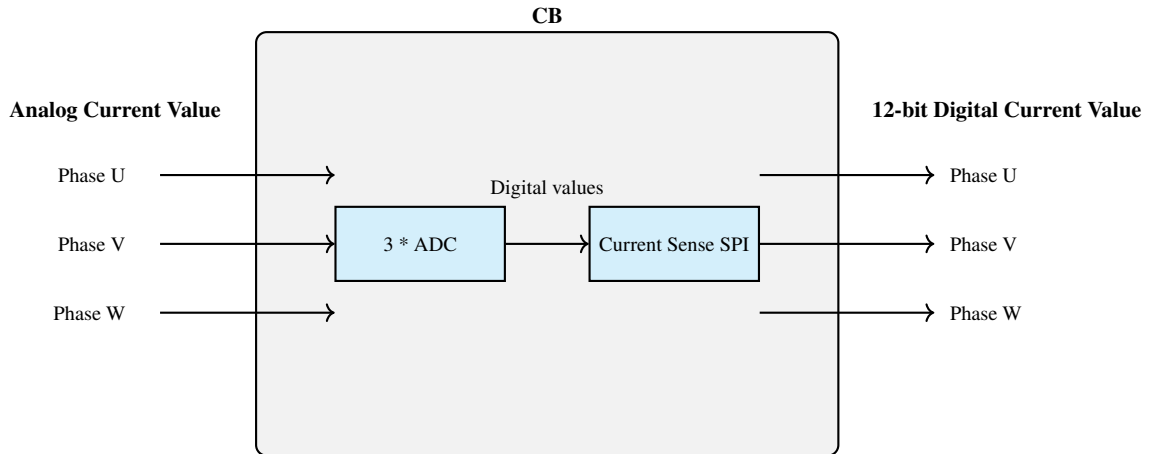


Figure 3.5: Simplified block representation of the three-phase current-sensing chain within the control board

3.2.2 OVC Detect

The OVC Detect module identifies overcurrent or short-circuit conditions on any of the three motor phases (U, V, or W) and subsequently disables the PWM outputs to protect the system. Detection thresholds are configurable via the AXI interface, allowing adjustment of both the lower limit for overcurrent and the upper limit for short-circuit detection. The OVC Detect module operates in conjunction with the Current Sense SPI module (refer figure 3.6) and is triggered whenever a new current measurement becomes available. It continuously compares the measured current values from all three phases to the configured threshold limits. Based on these comparisons, the module issues two signals: OVC status, which indicates the current fault condition, and OVC event, which serves as a fault notification.

Within the module, a finite state machine controls two primary error states.

- **Overcurrent:** The system transitions to this state if an overcurrent is detected in any of the three phases (U, V, or W). If the overcurrent persists for approximately 10 microseconds, the state advances to a short-circuit condition.
- **Short-Circuit:** In this state, the module sets the `ovc_event` output signal to logic '1', indicating a severe fault that requires immediate shutdown of the PWM.

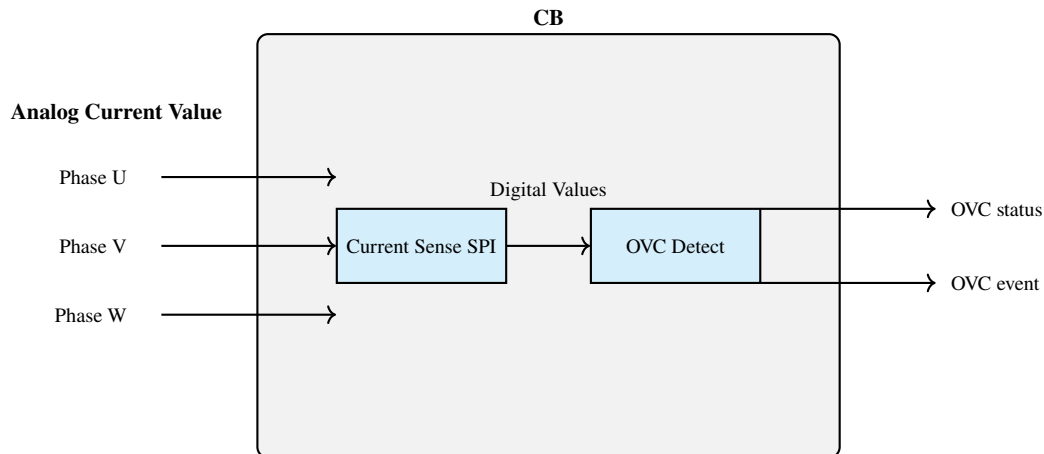


Figure 3.6: Simplified block representation of the Over-Current Detect path within the control board

3.2.3 PWM Control

The PWM_IP module (figure 3.7) generates PWM signals to drive the motor phases. It receives the desired frequency and duty cycle as inputs and produces corresponding PWM waveforms. The module supports three selectable switching frequencies: 16 kHz, 8 kHz, or 4 kHz, allowing adaptation to specific motor control requirements. It provides six PWM outputs, corresponding to the high-side and low-side signals for the three motor phases (U, V, and W). Although all six signals operate at the same switching frequency, each can be assigned an independent duty cycle, enabling flexible control and modulation for various operational scenarios.

In the current system, the PWM_IP module drives the motor hardware. The PWM signals are transmitted to the motor inverter stage, which generates phase currents in the motor windings. The control board's ADCs measure these phase currents, and the Current Sense SPI module processes the data for feedback and monitoring. Due to this interdependent configuration, several FPGA modules and hardware components rely on one another, necessitating the involvement of multiple modules during testing.

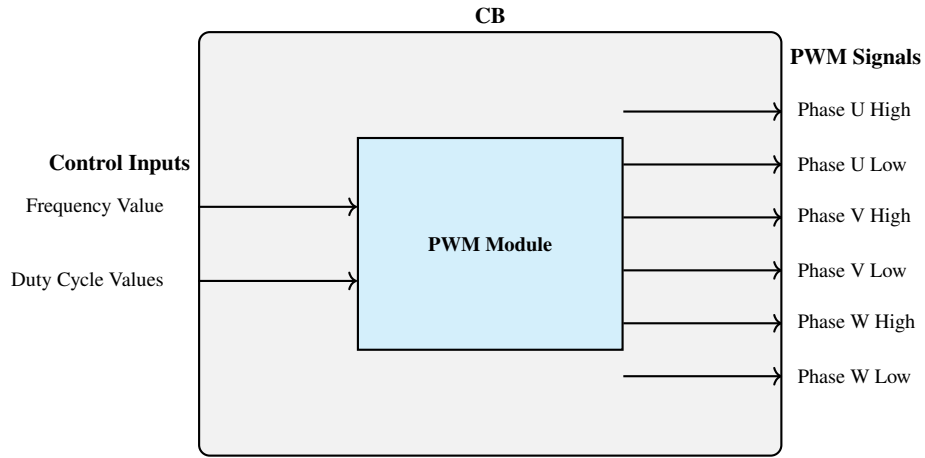


Figure 3.7: Simplified block representation of the PWM Generation Module within the control board

The interdependency among modules during verification complicates the isolation and testing of individual module functionality. Therefore, a dedicated FPGA Verification Platform is required to enable independent verification of FPGA modules and address the limitations identified in the problem statement [1.3]. The subsequent section presents the Requirement Analysis of the PoC for the proposed FPGA Verification Platform.

3.3 Requirement Analysis

The primary objective of the proposed verification platform is to emulate the necessary input stimuli for the specific DUT (FPGA modules), implemented on the control board, and to capture and analyze the corresponding feedback to determine whether each DUT operates as intended. To accomplish this, the verification platform must be implemented on an FPGA or SoC from the same family as the one used on the control board. This approach ensures architectural, timing, and interface compatibility, thereby facilitating accurate emulation and reliable verification outcomes. Several factors support selecting an FPGA over a microcontroller for this application:

- **Parallel Verification:** The system should support simultaneous execution of multiple verification tasks. This parallelism substantially reduces overall verification time and serves as a primary motivation for developing a dedicated platform.
- **Clock Flexibility:** The control board may operate at different clock frequencies across development stages, the verification platform must support multiple clock domains and remain compatible with all versions and configurations of the control board.
- **Multiple DUTs:** The control board integrates several FPGA modules that require functional verification. As additional features or modules are incorporated into the control board, the verification platform must adapt to emulate new input conditions and analyze the outputs of these expanded modules.
- **Extensibility:** The platform should be designed for scalability in both hardware architecture and verification logic, enabling straightforward integration of future test scenarios and additional FPGA components.

3.3.1 Enclustra Mercury XU5 + PE1 Evaluation board

The FPGA Verification Platform PoC employs the Enclustra Mercury XU5 SoM (figure 3.8) mounted on the Mercury+ PE1 Base Board (figure 3.9). This combination constitutes a high-performance platform built upon the heterogeneous architecture of the AMD Xilinx Zynq UltraScale+ MPSoC [14].

Mercury XU5 SoC Module: Processing Core

The Mercury XU5 SoM serves as the primary processing core and is based on the AMD Zynq UltraScale+ MPSoC [14]. This module provides a fully embedded processing system comprising the following key components:

- **Processing System:** The processing system integrates multiple ARM cores, including a quad-core Cortex-A53 for application management and a dual-core Cortex-R5 for real-time control [14].

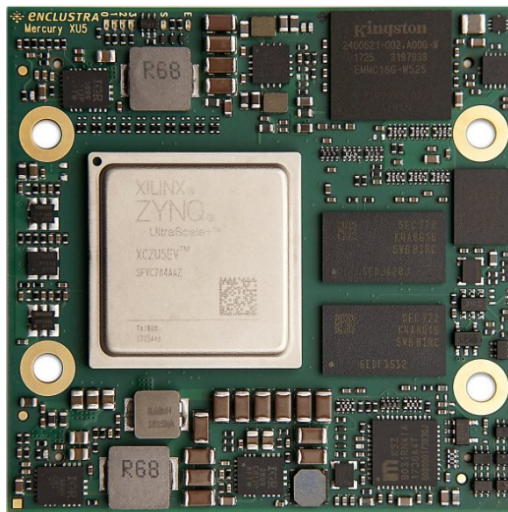


Figure 3.8: Mercury XU5 SoM Top View Module [14]

- **Programmable logic:** The PL fabric provides up to 256,000 LUT equivalents, which hosts the core FPGA modules [14].
- **Memory Architecture:** The module includes two independent DDR4 SDRAM channels with ECC support for the processing system and supports high-bandwidth data transfer [14].

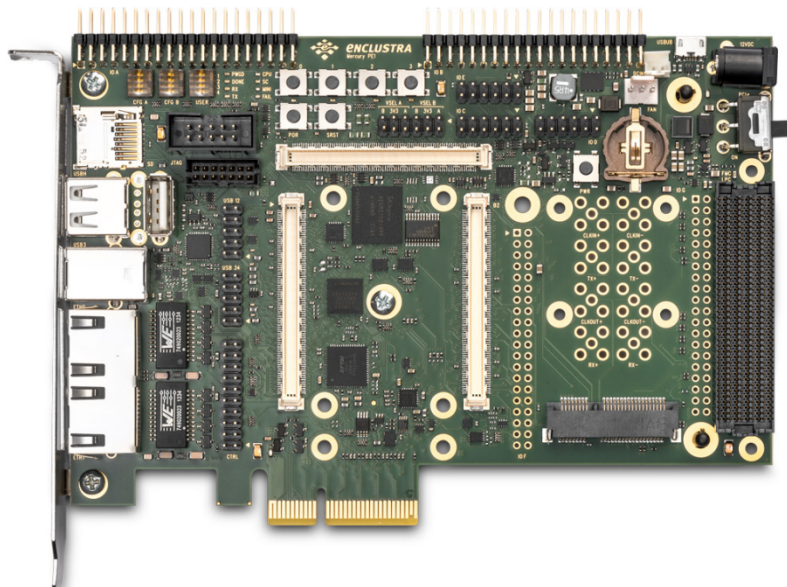


Figure 3.9: Mercury PE1 Board Top View [15]

Mercury+ PE1 Base Board: Interface and I/O

The Mercury+ PE1 Base Board functions as a versatile carrier, providing the necessary physical interfaces and I/O connections for the SoM. The primary features of the PE1 Base Board supporting the PoC include:

- **FMC Connectors:** The board accommodates up to two FMC Low Pin Count (LPC) connectors, enabling expansion for custom data acquisition or specialized DAC components [15].
- **Communication:** The board provides PCIe Gen2×4, USB 3.0, and multiple USB 2.0 interfaces, which are essential for host PC communication and debugging [15].
- **Flexibility:** The architecture supports use as either a standalone board or a PCIe card, which is advantageous during the rapid prototyping phase [15].

3.3.2 DAC80508 Evaluation Board

There is also a need for a DAC to convert the digital sine wave generated by the Verification Platform FPGA into an analog signal. Because the analog current values typically produced by physical motors are now emulated, this conversion is essential for verifying the CurrentSense SPI and OVC modules, as described in Section 3.2. The DAC80508-04EVM evaluation board (figure 3.10) facilitates the interfacing and evaluation of the DAC80508, a high-resolution, multi-channel DAC. The DAC80508 is part of the DACx0508 family of pin-compatible devices and is distinguished by its low power consumption, eight-channel architecture, and buffered voltage output [23].

1. Key Integrated Features [23]

- **Resolution :** The DAC80508 provides 16-bit resolution, resulting in highly granular analog signals and reduced quantization error and distortions, which is essential for smoother waveform reproduction.
- **Internal Reference :** The device integrates a 2.5 V precision internal reference, which typically eliminates the requirement for an external reference. This internal reference is accessible at the REF pin.

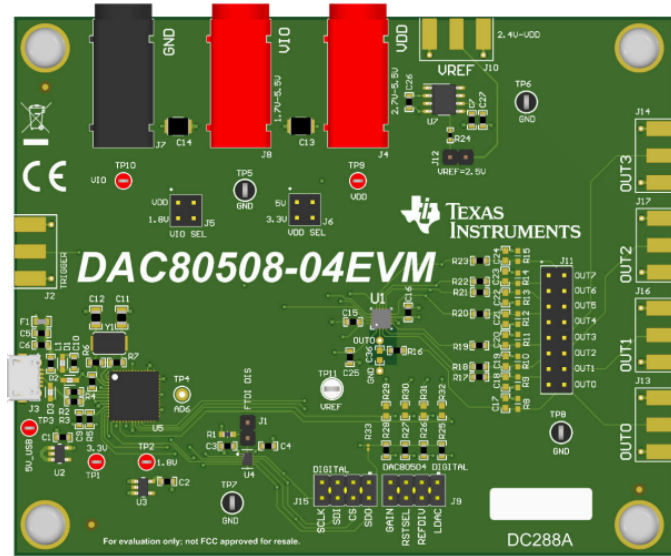


Figure 3.10: DAC80508-04EVM Evaluation board Top View [22]

- **Operating Range :** It operates reliably across a wide temperature range from -40°C to 125°C .
- **Output Configuration :** Each of the eight output channels (OUT0 to OUT7) supports individual gain configuration. User-selectable gain settings (x2, x1, or $\frac{1}{2}$) allow flexible output configuration. With the internal 2.5 V reference, these settings provide full-scale output voltages of 1.25 V, 2.5 V, or 5 V.

2. Digital Interface and Communication [23]

- **Serial Interface:** Communication with the DAC80508 is accomplished via a SPI compatible interface, supporting clock rates up to 50 MHz.
- **Supply Flexibility:** The digital interface (VIO pin) operates from 1.7 V to 5.5 V, enabling compatibility with a broad range of microprocessors and microcontrollers.
- **Data Integrity:** The device supports an optional CRC error check, which improves robustness in noisy environments by verifying SPI data integrity.

- **Multi-Device Operation:** The device supports daisy chain operation, allowing multiple DACx0508 devices to be connected together to minimize the number of required serial interface lines.

Several registers are available; however, the configuration, gain, status, and DACx registers are most pertinent to the objectives of this thesis.

1. CONFIG Register

Address: 0x03 | Default Value: 0x0000 | Access: Read/Write

The CONFIG register (refer table 3.1) governs essential device operations, such as alarm triggering mechanisms, serial data output configuration, and the power-down status of DAC channels.

2. GAIN Register

Address: 0x04 | Access: Read/Write

This register (see table 3.2) controls output gain and reference division. The default configuration depends on device type: DACx0508Z models use a gain of 1, whereas DACx0508M models use a gain of 2.

3. STATUS Register

Address: 0x07 | Default Value: 0x0000 | Access: Read-only

This register indicates the present status of the reference voltage.

4. DAC Channel Data Registers(DAC0- DAC7)

Address: 0x08 - 0x0F | Access: Read/Write

Each of the eight registers(Table 3.4) contains a digital value that determines the analog output of a DAC channel.

Owing to its high performance, compact package, and broad temperature range, the DACx0508 is suitable for applications such as optical networking, wireless infrastructure, industrial automation, and data acquisition systems [22]. The DAC80508-04EVM serves as a robust platform for evaluating an eight-channel, 16-bit-precision voltage-output DAC, low-drift internal reference, selectable output gains, and advanced digital capabilities, including ACS! error checking and per-channel power-down. The following section

Table 3.1: CONFIG Register (Address = 0x03, Reset = 0x0000) [23]

Bits	Field	Access	Reset	Function
15–14	Reserved	–	00	Internal factory use only.
13	ALM_SEL	R/W	0	This setting determines the function of the ALARM signal: a value of 0 indicates a CRC error, while a value of 1 indicates a reference-voltage alarm.
12	ALM_EN	R/W	0	Configures the SDO/ALARM pin to function either as a standard serial data output (0) or as an active-low, open-drain alarm output (1), requiring a 10 k Ω pull-up resistor. When enabled, the FSDO and DSDO signals are disregarded.
11	CRC_EN	R/W	0	A Value of 1 enables CRC checking of serial data, whereas a value of 0 disables this feature.
10	FSDO	R/W	0	Modifies the timing of SDO updates. When enabled, data is presented half a clock cycle earlier, specifically on the falling edge of SCLK.
9	DSDO	R/W	0	When enabled, the SDO line remains in high-impedance mode continuously.
8	REF_PWDWN	R/W	0	A value of 1 powers down the internal reference.
7–0	DACx_PWDWN	R/W	0	Each bit corresponds to a DAC channel (DAC0-DAC7). Setting a bit to 1 powers down the associated channel and connects its output to ground via an internal 1 k Ω resistor.

Table 3.2: GAIN Register (Address = 0x04) [23]

Bits	Field	Access	Default	Description
15–11	Reserved	–	0	Internal factory use only.
10	CLR_4TO7_MSK	R/W	0	(DACx0508C only) When 0, DAC4–7 respond to CLR pin; when 1, unaffected.
9	CLR_0TO3_MSK	R/W	0	(DACx0508C only) When 0, DAC0–3 respond to CLR pin; when 1, unaffected.
8	REFDIV_EN	R/W	0/1	Enables reference divider. When 1, reference voltage is halved.
7–0	BUFFx_GAIN	R/W	0/1	Per-channel output buffer gain. 0: Gain = 1; 1: Gain = 2.

Table 3.3: STATUS Register (Address = 0x07, Reset = 0x0000) [23]

Bits	Field	Access	Default	Description
15–1	Reserved	–	0	Internal factory use only.
0	REF_ALM	R	0	Indicates a reference alarm if V_{REF}/DIV drops below the minimum threshold.

Table 3.4: DACx Registers (Addresses = 0x08 to 0x0F) [23]

Bits	Field	Access	Default	Description
15–0	DACx_DATA	R/W	0x0000 / 0x8000	Holds the digital input code for each DAC channel (MSB-aligned straight binary). Default is zero for Z devices and midscale for M devices.

presents a detailed explanation of the Block Design for the FPGA Verification Platform PoC.

3.3.3 Block Design of the proposed FPGA Verification Platform (Proof of concept)

The Proof of Concept setup includes the Servo Drive Control Board, Enclustra Mercury XU5 Som, the PE1 board, and the DAC80508 Evaluation Board (refer figure 3.11). The Zynq UltraScale+ MPSoC on the Mercury XU5 generates static and dynamic input signals, which trigger the DAC to produce the analog signals required for CurrentSense SPI verification as mentioned in section 3.2. These analog signals serve as simulated motor current feedback and are connected to the ADCs on the Hot Side of the control board. This configuration replaces the physical motor with a digitally controlled analog system, enabling closed-loop testing of the sensing and protection modules.

During each verification test, the SoC transmits specific commands to the Control Board via a UART interface. These commands specify the test parameters and input values for the FPGA modules under test (DUTs). Upon receiving a command, the Control Board FPGA processes the instruction, executes the test, and returns the DUT's output data as feedback to the Verification SoC via the same UART channel.

This closed communication loop between the Verification SoC and the Control Board enables detailed analysis of DUT responses. By comparing the transmitted input data with the received feedback, the SoC verifies whether the DUT operates as intended under test conditions. This approach supports a systematic and reliable verification process, facilitating accurate testing of each FPGA module and accommodating future design modifications. The block design of this PoC demonstrates the feasibility of establishing an independent FPGA verification setup capable of simulating real hardware interactions without requiring the complete motor control system.

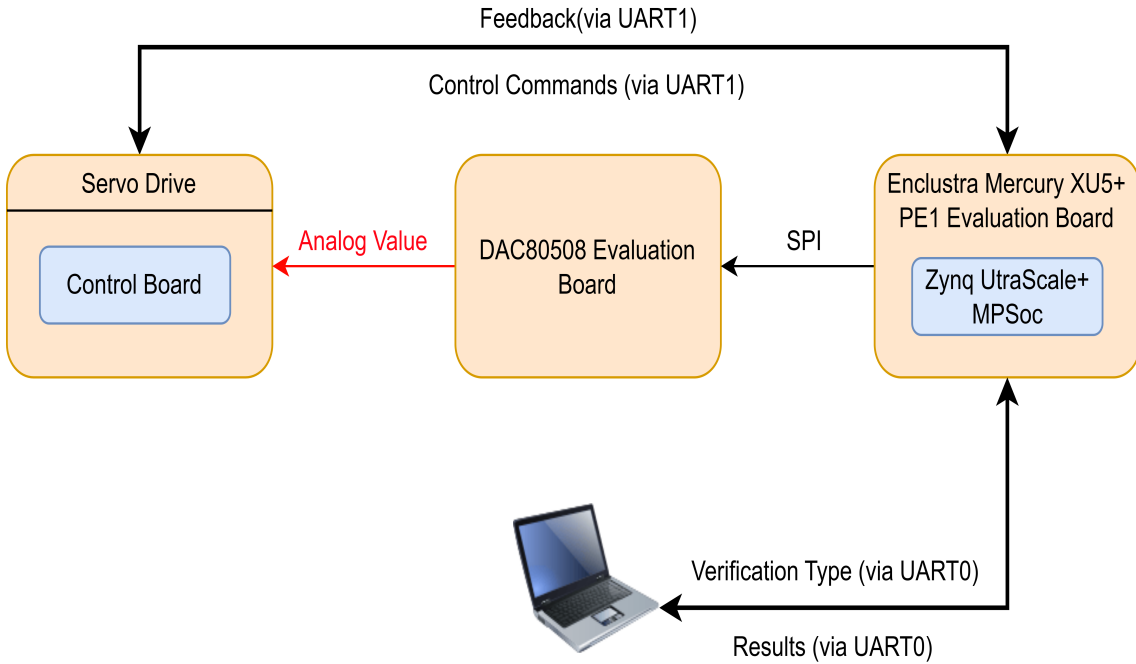


Figure 3.11: Block Design of the FPGA Verification Platform PoC

3.4 Related Work and Case studies in FPGA and HIL Verification

The presented PoC establishes a self-contained verification setup capable of generating analog signals, simulating motor feedback, and testing sensing and protection modules via a fully digital command-and-feedback loop. To contextualize this contribution within the broader research field, existing methods addressing similar verification challenges using FPGA-based, HIL, or FIL techniques are reviewed. The review highlights key examples from power HIL, FPGA-in-the-Loop verification, and analog-interface emulation. Current gaps in the literature are identified, and it is demonstrated how this PoC advances the field by providing low-power, closed-loop, and fully programmable verification for CurrentSense SPI and protection logic.

1. "An FPGA-in-the-Loop Approach for HDL Motor Controller Verification" [27]

This paper presents the FIL simulation methodology for verifying motor control systems that require concurrent simulation of both the control algorithm and the motor. Previous approaches that relied on prerecorded motor responses frequently yielded inaccurate results. In the FIL approach, an FPGA is programmed with the HDL design, and a simulated motor response is used to interact with the controller in a closed-loop configuration. This methodology was applied to implement, test, and validate both the Field-Oriented Control (FOC) algorithm and the Voltage-by-Frequency (V/f) control strategy for a Permanent Magnet Synchronous Machine (PMSM). The systems were designed in MATLAB Simulink and translated to VHDL using HDL Coder. FIL simulation results closely matched software simulation results, indicating that prior discrepancies originated in the simulation method rather than the controller design. This approach may facilitate future controller switching for optimization or fault tolerance through Dynamic Partial Reconfiguration.

2. "FPGA-based Real-Time Hardware-In-the-Loop Simulator of a Mini Solar Power Station" [9]

The article describes the development of a real-time HIL simulator that employs an FPGA to model a miniature solar power station, specifically a solar panel-fed battery charger. The primary motivation for implementing HIL is to enable safe, rapid testing of the digital controller unit (CU), thereby reducing the risks and costs of field testing and addressing parameter inaccuracies inherent in low-power laboratory models. The HIL simulator replaces the high-power components of the system with computational models implemented in MATLAB/Simulink and running on the FPGA. These simulated components interface with the control unit through physical connections, including analog and digital channels. The solar photovoltaic system model incorporates the effects of temperature and radiation by utilizing normalized characteristics stored in block RAMs as look-up tables for real-time computation. To generate the analog signals (u_S , u_A , i_L) required by the control unit's DSP, the simulator employs sigma-delta DAC. The workflow leverages HDL synthesis from Simulink to reduce the need for manual code optimization.

3. **"FPGA-Based Real-Time Simulation of Nonlinear Permanent Magnet Synchronous Machines for Power Hardware-in-the-Loop Emulation Systems"** [28]

A real-time simulation system based on FPGA technology is presented for modeling nonlinear Permanent Magnet Synchronous Machines in Power HIL emulation systems. PHIL is designed to replace conventional, bulky motor-load testbeds with a flexible platform that can emulate various electric machines. The proposed machine model incorporates complex nonlinear phenomena, including magnetic anisotropy, iron saturation, and dynamic cross-coupling between the d- and q-axes. The primary contribution is a custom, high-performance signal-processing system that solves the machine's differential equations at 1.5 MHz. Additionally, the model computes the counter-voltage required by the emulation converter, ensuring that the system accurately replicates the current slopes of an actual machine regardless of the inductance in the coupling network. The implementation employs fixed-point representation generated using MATLAB/Simulink and HDL Coder, achieving high accuracy with a relative error of less than 0.01% in the current simulation compared to the floating-point model.

4. **"FPGA-synthesizable Filter Model Based Bitstream DAC for Hardware-in-the-Loop Simulators"** [25]

The article examines methods to enhance the analog output quality of FPGA-based HIL simulators, focusing on addressing the limited number of available FPGA pins. Traditional Sigma-Delta (Σ - Δ) DAC can be implemented on FPGAs using a single pin per output, however, they require an analog low-pass filter, which restricts signal bandwidth and introduces phase delay. This limitation is particularly problematic for fast current control loops that frequently utilize triangular waveforms. The study proposes a novel Filter Model-Based DAC architecture. The FMB DAC incorporates a digital representation of the analog filter within the FPGA to compensate for the low-pass filtering effect. A controller generates the bitstream by comparing the target input signal with the digital filter model's output. Experimental results demonstrate that, unlike conventional Σ - Δ DACs, the FMB DAC accurately reproduces a 10 kHz triangular waveform with minimal phase delay and a significantly faster settling

time. Additionally, it supports filter bandwidths up to 30 kHz. This performance improvement, however, comes at the cost of substantially increased FPGA resource usage, requiring approximately four to five times more resources than a standard Σ - Δ DAC.

3.4.1 PoC Solutions to Identified Research Gaps

- **PHIL complexity vs verification scope**

PHIL offers accurate power-level emulation, however it requires emulation converters, coupling networks, and precise management of converter dynamics. These requirements render PHIL expensive, complex, and potentially unsafe for routine module-level testing. Although existing studies demonstrate the advantages of PHIL, they also emphasize its logistical and cost challenges. [28]

This PoC utilizes the Mercury XU5 System-on-Chip to generate both static and dynamic signals. A DAC80508 evaluation board converts these digital signals into analog currents or voltages, which are subsequently transmitted to the Control Board's ADCs. This configuration replicates the measurement behavior observed by the DUT without requiring an emulation converter or power-level coupling network. It preserves essential sensing capabilities while significantly reducing complexity, cost, and safety risks compared to PHIL. Consequently, early-stage module testing becomes more feasible and secure.

- **Incomplete analog-chain verification in FIL/HIL workflows vs End-to-end verification that includes the analog acquisition chain**

FIL verifies HDL controller functionality and timing; however, it typically does not assess the analog sensing and acquisition chain, including DAC fidelity, analog front-end filtering, ADC timing, CurrentSense IC interaction, or SPI readout. Consequently, sensor-level issues such as ADC sampling shifts, CurrentSense SPI timing problems, or analog filtering effects may remain undetected until full hardware integration [27].

The PoC stimulates the Control Board ADC inputs with actual analog waveforms generated by the DAC and evaluates the DUT responses. This approach tests the entire measurement path, encompassing DAC output fidelity, analog filtering and grounding, ADC sampling, CurrentSense IC and SPI behavior, and downstream FPGA protection logic. It extends FIL's logic-level validation to include the complete sensor path.

- **Lack of targeted FPGA module verification vs Automated command/feedback FPGA module verification**

Most existing research on HIL and FIL focuses on motor models, converter emulation, or controller parity. However, these studies do not directly address the FPGA modules as the primary verification target[9] .

In the proposed PoC, the SoC transmits verification commands and parameters to the Control Board via UART1. The FPGA on the Control Board executes the DUT and returns the feedback. The SoC subsequently verifies whether the DUT's feedback match the expected results and determines the test outcome. This automated command-and-feedback loop enables repeatable, parameterized test sequences and facilitates scalable regression-style verification for individual FPGA modules. This approach represents an advancement over the manual HIL setups described in previous research.

4 Implementation

This chapter outlines the development process of the PoC / FPGA Verification Platform for this thesis. All components, including hardware designs, programmable logic modules, signal generators, analysis tools, and software, were developed from first principles. No pre-existing verification frameworks, IP extensions, or platform structures were available for adaptation. Each functional block discussed in the following sections, ranging from digital stimulus generation to closed-loop signal evaluation, represents original work created specifically for this proof-of-concept. The chapter provides a comprehensive account of the implementation process prior to evaluating system performance.

4.1 Current Sense Verification

Within the Current Sense verification system, the Current Sense path functions as a black box. The ADC on the control board receives analog inputs, either static or dynamic, corresponding to each motor phase as mentioned in the section 3.2.1. The DAC80508 generates these analog signals. A high-level block diagram of the Current Sense Verification Closed-Loop architecture is provided in the figure 4.1.

The system comprises three motor phases (U, V, and W) and supports two types of verification tests: static and dynamic. Accordingly, six control commands are defined.

1. `phu_s_a1` – Phase U Static Test Axis 1
2. `phv_s_a1` – Phase V Static Test Axis 1
3. `phw_s_a1` – Phase W Static Test Axis 1
4. `phu_d_a1` – Phase U Dynamic Test Axis 1
5. `phv_d_a1` – Phase V Dynamic Test Axis 1
6. `phw_d_a1` – Phase W Dynamic Test Axis 1

Each control command specifies the phase under test and the verification type (static or dynamic). These commands are transmitted to the control board, where the test_fsm module interprets the command and generates the corresponding feedback.

The PS UART interface on the Enclustra Mercury XU5 SoC receives and parses the control commands. The relevant data is transferred to the PL through the AXI GPIO interface. A custom MUX module processes the AXI GPIO inputs and handles DAC initialization and configuration before data transmission.

- For static verification, a single 16-bit value is written to the selected DAC to emulate a constant analog signal.
- For dynamic verification, a 16-bit sine wave lut is written to the DAC to emulate an analog sinusoidal waveform representative of motor feedback.

Three of the eight available DAC channels on the DAC80508 evaluation board are used, each corresponding to one motor phase (U, V, and W). The appropriate DAC channel is selected and updated with the required data based on the received command.

The Direct Digital Frequency Synthesis (DDFS) module generates digital sine wave samples for dynamic verification. These samples are transmitted to the DAC through an SPI interface module. The ADC on the control board measures the resulting analog output and, according to the test configuration, sends feedback to the Enclustra board via a secondary UART interface.

During static tests, a single 12-bit ADC value is returned. For dynamic tests, a FIFO containing 2048 12-bit samples is transmitted. The PS then performs the following verification steps.

- In static tests, the received 12-bit ADC value is compared with the corresponding 16-bit reference value transmitted from the Enclustra board.
- In dynamic tests, a FFT is performed on the received 12-bit FIFO data to verify the frequency content. If the FFT result confirms the expected 1 kHz sine wave, the dynamic response is considered valid.

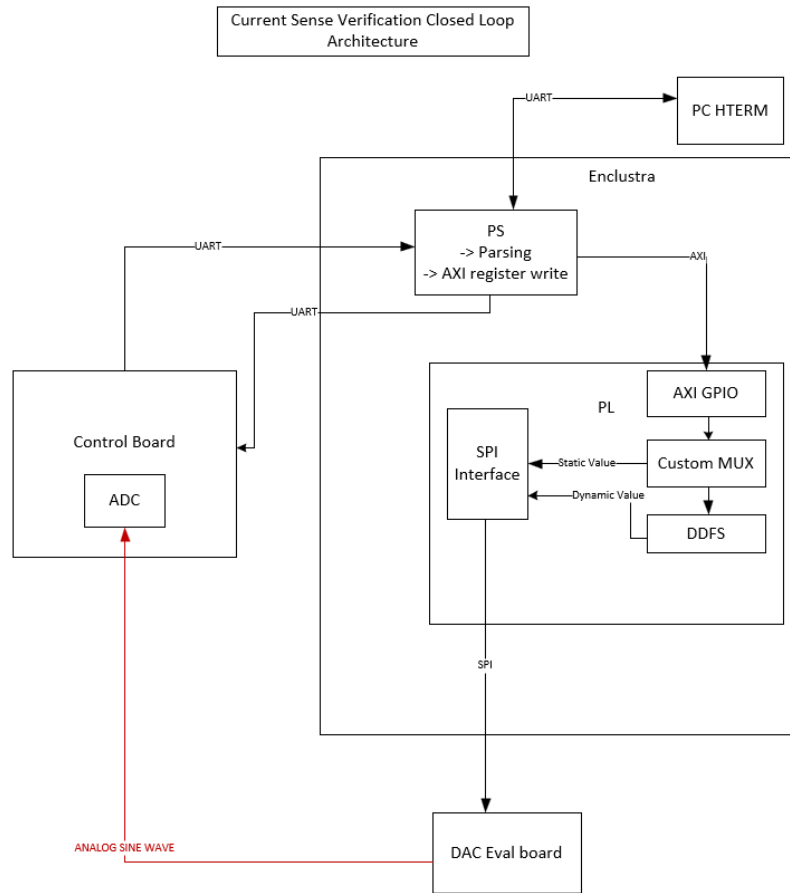


Figure 4.1: Block Design of the Current Sense Verification

In a nutshell, to facilitate Current Sense Verification, the system interprets user commands and transmits the appropriate data to the DAC channels. A dedicated Custom MUX module manages this process, acting as the interface between PS-generated control signals and the DAC configuration logic. The following section details the design and operation of this module.

4.1.1 Custom MUX algorithmn

The Command Parser, implemented in the PS, writes the relevant data parsed from the control commands to the AXI GPIO. This data is then transferred to the Custom MUX module within the FPGA, which configures the DAC on the DAC80508 evaluation board.

A state machine (figure 4.2) is implemented to transmit the configuration command **0x30100** to the DAC80508 device. By default, the evaluation board utilizes a 2.5 V internal reference, however, an external reference may be used by disabling the internal reference via the CONFIG register [23].

The DAC requires a 24-bit hexadecimal command format. The first nibble indicates the operation type, with '0' representing a read and '1' representing a write. The second nibble specifies the register address, while the remaining 16 bits contain the data to be written or read. The command **0x30100** is sent to the CONFIG register (see Table 3.1) to power down the internal reference and enable an external 3.3 V reference source. In the following state, this configuration is verified by issuing the command **0x13000**, which allows the value to be read back through the MISO lines as confirmation.

Similarly, the Gain register is updated with the command **0x401FF**, configuring the DAC so that the output gain is doubled and the reference voltage is internally divided by two (see Table 3.2). Depending on the control command, if static verification, a certain static Value is written to a DAC register, if dynamic Verification, the DDFS is triggered which starts to generate digital sine wave from a LUT of 16 bit 1024 samples. The system executes one of two actions depending on the control command.

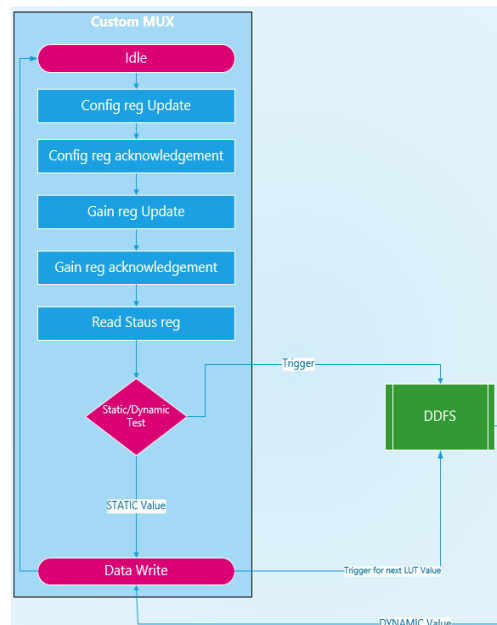


Figure 4.2: Current MUX algorithm

The Custom MUX selects the verification method, while generating dynamic test signals requires a precise digital waveform source. For this purpose, the system incorporates a Direct Digital Frequency Synthesis module that produces high-resolution sine waves. The following subsection details the DDFS architecture, configuration parameters, and the process for generating the 1 kHz test signal.

4.1.2 Direct Digital Frequency Synthesis

Direct Digital Frequency Synthesis is a widely recognized technique for generating sinusoidal signals in applications that require high frequency resolution, rapid frequency and phase transitions, and high spectral purity. DDFS systems generate periodic, discrete-time waveforms at specified frequencies. Although the output waveform is typically a sine wave, DDFS can also produce saw-tooth, triangle, square, or other periodic waveforms. The primary components of a DDFS system are illustrated in Figure 4.3.

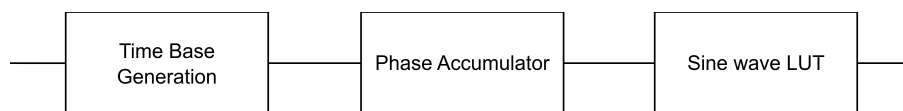


Figure 4.3: Direct Digital frequency Synthesis

- **Time Base Generation** → The time-base module receives a 192 MHz system clock generated by the clocking wizard from a 100 MHz external oscillator. This module periodically enables the phase accumulator, allowing the DDFS to output a sine wave at the specified frequency. The counter increments with each clock cycle and generates an enable pulse when it reaches a predefined threshold.

```

1  time_base_counter : process(clk, rst_n)
2  begin
3      if rising_edge(clk) then
4          if rst_n = '0' then
5              time_base <= (others => '0');
6          else
7              if ddfs_start = '1' then
8                  if time_base = parameter - 1 then
9                      time_base <= (others => '0');
  
```

```

10         else
11             time_base <= time_base + "1";
12         end if;
13     end if;
14 end if;
15 end if;
16 end process time_base_counter;

```

- **Phase Accumulator** → The phase accumulator generates the phase angle used to index into the sine look-up table. By adding the frequency-tuning word (FTW) at each enable pulse it determines both the output frequency and phase resolution.

```

1  phase_acc_stimulus : process(clk)
2  begin
3      if rising_edge(clk) then
4          if rst_n = '0' then
5              q_reg <= (others => '0');
6          elsif enable = '1' then
7              q_reg <= q_reg + unsigned(ftw);
8          end if;
9      end if;
10 end process phase_acc_stimulus;

```

- **Sine Wave LUT** → The LUT translates the accumulated phase value into a corresponding sine amplitude. It consists of 1024 samples, each 16 bits wide, representing a complete 360° sine wave.

A system clock frequency of 192 MHz is selected for the verification path. The goal is to synthesize a 1 kHz sine wave using the direct DDFS. This target frequency is chosen based on the current verification setup (see Section 1.3), in which the motor drives are capable of generating phase-current feedback signals at 500–800 Hz. To achieve a 1 kHz output, the time-base generation module must trigger the phase accumulator once every 192 clock cycles. Assuming the use of 1,000 LUT samples with 16-bit amplitude resolution, the output frequency can be calculated as follows:

$$\begin{aligned}\frac{192 \times 10^6}{192(Timebase)} &= 10^6 \\ &= \frac{10^6}{1000(LUTSamples)} \\ &= 1\text{ KHz}\end{aligned}$$

However, selecting 1,000 samples necessitates a 10-bit phase accumulator, which inherently provides 1,024 addressable entries. Because the phase accumulator directly indexes the sine LUT, the table size must be expanded to 1,024 samples. To maintain consistency with this adjustment and preserve the intended output frequency, the number of clock cycles in the time-base generation module is correspondingly set to 188, as reflected below:

$$\begin{aligned}\frac{192 \times 10^6}{188(Timebase)} &= 1021276.596 \\ &= \frac{10^6}{1024(LUTSamples)} \\ &= 997.340\text{ Hz}\end{aligned}$$

Each LUT sample must be transferred to the DAC at 1 μ s intervals via SPI. This interval is justified because a single SPI transaction, from the chip-select signal going low to returning high, requires approximately 600 to 700 ns. Consequently, updating the DAC every 1 μ s is feasible and supports the reliable generation of a 1 kHz sine wave.

$$\text{Time per clock cycle} = \frac{1}{192 \times 10^6} = 5.208333 \times 10^{-9} \text{ sec}$$

$$\text{Time for 188 cycles} = 5.208333 \times 10^{-9} \text{ sec} \times 188$$

$$\text{Time for complete 1024 LUT samples} = 9.791666 \times 10^{-7} \times 1024$$

$$\text{Hence the frequency of the DDFS output} = \frac{1}{1.002 \times 10^{-3} \text{ sec}} = 997.340 \text{ Hz}$$

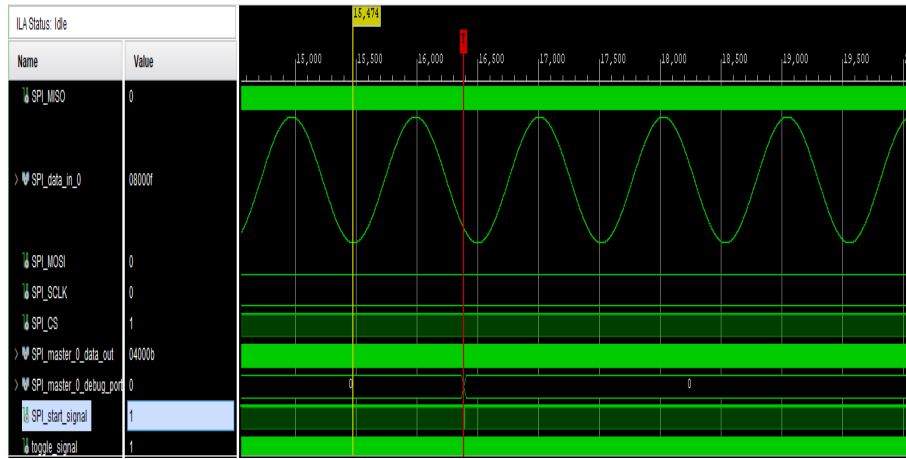


Figure 4.4: Sine wave generated from the DDFS

The DDFS operates effectively and delivers sine-wave samples to the DAC in a timely manner (refer figure 4.4) only when the SPI interface functions reliably. Since both static values and generated lookup table samples must be transmitted rapidly, selecting appropriate system and SPI clock speeds is essential. The following section details the rationale for selecting a 192 MHz system clock and demonstrates how this choice satisfies the SPI timing and throughput requirements.

4.1.3 Clock period selection for SPI

The selection of a 192 MHz system clock for the verification setup is primarily determined by the timing and performance requirements of the SPI interface utilized with the DAC80508. The DAC receives either a constant value for static verification or continuous

16-bit sine-wave samples for dynamic verification, requiring the SPI module to operate at a speed sufficient to maintain uninterrupted data flow. This design supports two SPI clock settings, 48 MHz and 32 MHz. The system clock must support straightforward, integer-based divisions to generate these SCLK frequencies.

A 192 MHz base clock enables straightforward, flexible generation of the required SPI clocks. A 48 MHz SCLK is achieved by dividing by 4, while a 32 MHz SCLK is obtained by dividing by 6. Employing whole-number divisions eliminates the need for fractional clock generation, which can introduce jitter, phase noise, and timing complications. Minimizing clock edge jitter is particularly critical when interfacing with precision analog components such as the DAC80508, as timing inconsistencies can adversely affect output accuracy.

The selection of this clock is closely associated with the dynamic verification process. If a slower system clock were selected, achieving high SPI rates with deterministic timing would be considerably more difficult and could compromise the integrity of the reconstructed analog waveform. Following the configuration of the Custom MUX,DDFS, clocking architecture, and SPI timing strategy for the Current Sense verification path, the implementation advances to the second major subsystem of the verification framework: PWM Verification. The following section details the design methodology, signal validation procedures, and verification steps employed to evaluate the PWM IP module (refer 3.2).

4.2 PWM Verification

The objective of PWM verification is to validate the operational behavior of the PWM_IP block (see Section 3.2.3) implemented on the control board. The PWM counter operates in an up–down counting mode with a 64 MHz carrier frequency. Both the PWM period and duty-cycle values are configured by the user through the verification platform.

4.2.1 PWM_IP Inputs and Control Commands

The PWM compare value has a resolution of 13 bits. For UART transmission, this value is packed into a 16-bit (2-byte) data field. The duty-cycle vector `pwm_uvw_compare` is

39 bits wide and consists of three 13-bit segments, each corresponding to an individual phase. For UART transmission, the 39-bit vector is transferred as 5 bytes.

- Bits 0-12 represent Phase U.
- Bits 13-25 represent Phase V.
- Bits 26–38 represent Phase W.

To configure the PWM module, the control board requires the following control commands:

PWM Frequency Configuration

- Command (ASCII), for example: **pwm_f_a1**, which sets the PWM frequency for Axis 1.
- Data (HEX, 2bytes), for example: **07D0**, corresponding to a 16 KHz PWM period.

PWM Duty-Cycle configuration

- Command (ASCII), for example: **pwm_d_a1**, which sets the duty cycle for Axis 1, phases U, V, and W.
- Data (HEX, 5 bytes), for example: **0FA07D03E8**, representing a 50 percent duty cycle on all three phases.

After each command (pwm_f_a1 or pwm_d_a1), the state machine in the control board waits until the associated data is successfully written to the corresponding PWM IP register. Once the duty-cycle values have been transmitted and written for the first time, the PWM output is enabled. Following table explains possible frequency setting for the PWM IP:

Table 4.1: PWM frequency settings

PWM Frequency	PWM Cycle	pwm compare [hex]	pwm compare [dec]	Counter Steps (Full Cycle)
4 kHz	250 μ s	1F40	8000	16000
8 kHz	125 μ s	0FA0	4000	8000
16 kHz	62.5 μ s	07D0	2000	4000

Formula for setting the PWM frequency ($f_c = 64$ MHz):

$$\text{period_compare}[\text{dec}] = \frac{f_c}{2 \times f_{\text{PWM}}}$$

Formula for setting the PWM duty cycle for each phase ($X = \text{U, V or W}$):

$$\text{pwm_X_compare}[\text{dec}] = \frac{\text{duty}_{\%}}{100} \times \text{period_compare}[\text{dec}]$$

When measuring the PWM duty cycle on the FPGA Verification platform, it is necessary to account for the PWM dead time between high and low signals. On the control board, the dead time is fixed at $1 \mu\text{s}$. For each phase, the control board generates two complementary PWM signals: one for the high-side switch and one for the low-side switch. These signals are designed so that when the high-side switch is ON, the low-side switch is OFF, and vice versa. The inclusion of dead time between switching events ensures that both switches are never ON simultaneously, thereby preventing a direct short circuit across the DC bus, a condition known as shoot-through. Shoot-through can result in excessive current, damage to power devices, and potential destruction of the control board.

The dead time results in a different value of the duty cycle between CB settings and FPGA Verification Platform measuring. Calculation formula for expected measured duty cycle by a dedicated set duty cycle value:

$$\text{duty}_{\text{cycle}}[\%] = \left(\frac{1}{f_{\text{PWM}}} \times \frac{\text{duty}_{\text{set}}[\%]}{100} - t_{\text{dead}} \right) \times \frac{1}{f_{\text{PWM}}}$$

An example illustrating a PWM frequency of 16 kHz with duty cycles of 50% for phase U, 30% for phase V, and 80% for phase W is provided in the Appendix A. The accompanying

figure 4.5 illustrates the resulting PWM waveforms, showing the varying duty cycles for each phase and highlighting the $1\ \mu\text{s}$ dead time between high and low transitions.

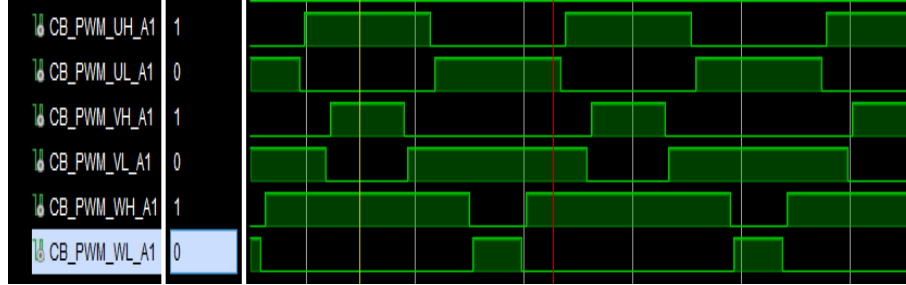


Figure 4.5: The PWM Signals generated at the CB with variable duty-cycle values

4.2.2 Block Design of PWM Verification

The closed-loop PWM verification architecture (refer figure 4.6) is implemented to validate the functionality and accuracy of PWM signals generated by the control board. In this configuration, control commands are transmitted to configure specific PWM frequency and duty cycle values. The PS of the MPSoC parses these commands and communicates the relevant data to the CB via UART. The control board requires 2 bytes of data for the `pwm_f_a1` command, which sets the PWM frequency, and 5 bytes for the `pwm_d_a1` command, which sets the duty cycle.

The control board generates six PWM output signals, corresponding to three phases: Phase U High and Low, Phase V High and Low, and Phase W High and Low. These signals are interfaced directly with the FPGA's PL section for real-time analysis.

Within the PL, PWM Analyzer modules process the input PWM signals and calculate both the total PWM period and the effective duty cycle count in clock cycles for each phase. These values are communicated to the PS, where further processing determines the actual PWM frequency and duty cycle for each signal.

The DeadTime Capture module in the PL monitors pairs of PWM signals from the same phase (high and low) to determine dead time intervals. The captured dead-time counts are combined into a single 32-bit register that encodes the time intervals (clock cycles)

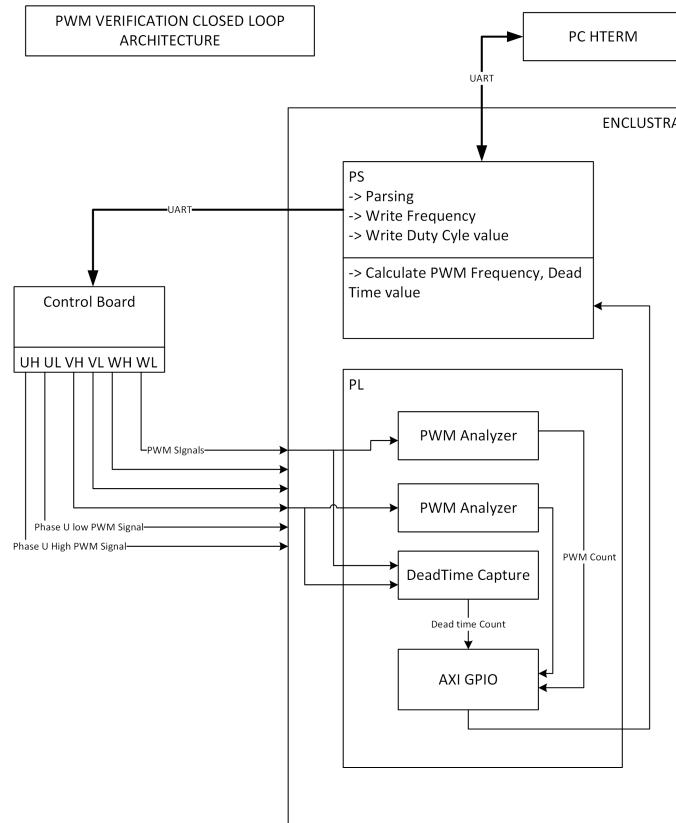


Figure 4.6: Block diagram depicting the closed-loop PWM verification architecture, which demonstrates the interaction among the control board, SoM, and FPGA based signal analysis modules

between high-to-low and low-to-high transitions. This information is transmitted to the PS via the PWM Capture AXI module to calculate the dead time for each phase.

This closed-loop verification approach enables precise monitoring of PWM signal characteristics and inter-phase dead times, establishing a robust framework for validating the performance and timing integrity of the PWM IP block on the control board.

4.2.3 PWM Analyzer

The PWM Analyzer FPGA subsystem measures the characteristics of an incoming PWM signal, specifically its period and duty cycle, by counting high-speed system clock cycles between key signal transitions. The module identifies consecutive rising edges of the PWM

signal to determine the total period, while the number of clock cycles between the rising and falling edges is used to compute the duty-cycle duration. Both measurements are accumulated using 16-bit counters, as illustrated in Figure 4.7. In addition to these edge detectors, a free-running counter runs continuously while the system is active, enabling the logic to compute differences between edge timestamps. These calculated values are written to AXI-accessible registers, from which the PS can retrieve them. The PS converts these clock-cycle counts into user-interpretable PWM parameters such as frequency and duty cycle.



Figure 4.7: PWM Analyzer RTL Block Diagram

The verification platform operates at a 192 MHz system clock, whereas the PWM signals under analysis arrive at significantly lower frequencies (16 kHz, 8 kHz, or 4 kHz) and originate from an independent clock domain. Direct sampling of these asynchronous signals introduces the risk of metastability, potentially leading to unpredictable behavior if input transitions violate flip-flop timing requirements. To ensure reliable operation, the incoming PWM signal must be safely synchronized to the internal 192 MHz clock.

To address this issue, the PWM Analyzer employs a three-stage flip-flop synchronizer, a well-established technique for transferring asynchronous signals into a synchronous clock domain. The synchronizer captures the incoming PWM signal via three sequentially clocked registers, thereby significantly reducing the likelihood that metastability propagates into the measurement logic. A snippet of this synchronizer implementation is provided below. This approach ensures that subsequent period and duty-cycle measurements are performed on a clean, stable, and fully synchronized version of the PWM waveform.

```
1 | sync_pro : process(clk, rst_n)
```

```
2      begin
3          if rising_edge(clk) then
4              if rst_n = '0' then
5                  PWM_sync_0 <= '0';
6                  PWM_sync_1 <= '0';
7                  PWM_sync   <= '0';
8              else
9                  PWM_sync_0 <= PWM_in;
10                 PWM_sync_1 <= PWM_sync_0;
11                 PWM_sync   <= PWM_sync_1;
12             end if;
13         end if;
14     end process sync_pro;
```

The PWM signals originate from the control board, they are inherently asynchronous relative to the verification FPGA's internal clock domain. A further challenge occurs when the incoming PWM signal becomes inactive for an extended period. Although the edge counters within the PWM Analyzer stop incrementing when no additional transitions are detected, the previously stored count values remain unchanged. If the system reads these outdated values, the module may incorrectly indicate that a valid PWM signal continues to exist. This behavior can result in misleading diagnostics or incorrect parameter estimations within the processing system.

A watchdog mechanism is integrated into the PWM Analyzer architecture to address this issue. The watchdog detects extended inactivity on the synchronized PWM input and ensures that all measurement counters are reset when the signal is absent. The watchdog logic includes an internal counter that increments whenever no PWM edges, either rising or falling, are observed. While valid PWM activity persists, this counter is periodically cleared. If the inactivity counter reaches a predefined threshold, the system interprets this as a loss of PWM input. Upon exceeding this threshold, the watchdog resets the period and duty-cycle counters, thereby clearing any outdated measurement data. This approach ensures that the PWM Analyzer does not retain obsolete timing values and provides an

accurate indication of signal presence to the remainder of the system. The following code snippet illustrates a portion of the watchdog implementation.

```

1  watchdog_pro : process(clk,rst_n)
2  variable watchdog_counter : unsigned (16 downto 0)
3  := (others => '0');
4  begin
5      if rising_edge(clk) then
6          if rst_n = '0' then
7              watchdog_counter := (others => '0');
8              watchdog_active <= '0';
9          else
10             if ((redge_pwm = '1') or (fedge_pwm = '1')) then
11                 watchdog_counter := (others => '0');
12                 watchdog_active <= '0';
13             elsif watchdog_counter < WATCHDOG_LIMIT then
14                 watchdog_counter := watchdog_counter + 1;
15                 watchdog_active <= '0';
16             else
17                 watchdog_counter := WATCHDOG_LIMIT;
18                 watchdog_active <= '1';
19             end if;
20         end if;
21     end if;
22 end process watchdog_pro;

```

4.2.4 Dead Time Capture

Figure 4.8 presents the architecture of the Dead Time Capture module implemented on the FPGA. The module monitors complementary PWM signals, specifically the high-side and low-side drive signals, and determines the dead time interval between their transitions. It detects the relevant signal edges on both inputs and measures the interval during which neither signal is active, a parameter critical to ensuring safe switching in power electronic systems.

The module stores the calculated dead time in a 32-bit output register, which the PS accesses via the AXI interface. This configuration enables the PS to log, analyze, or process the timing data in a manner analogous to its handling of data from the PWM Analyzer module.

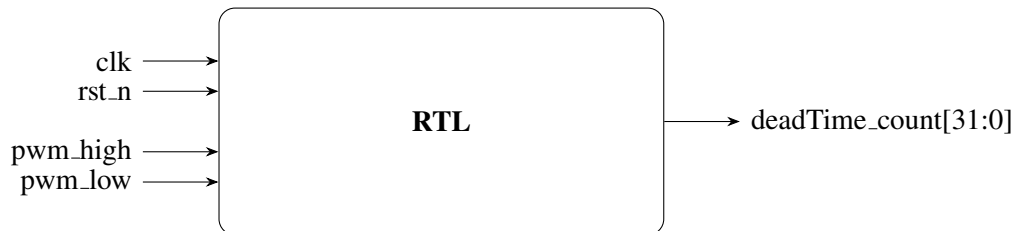


Figure 4.8: Dead Time Capture RTL Block Diagram

The Dead Time Capture module employs synchronization and signal-validation methods identical to those of the PWM Analyzer block to ensure reliable operation. A three-stage flip-flop synchronizer safely transfers external PWM signals into the FPGA clock domain, while a watchdog unit continuously monitors signal activity. As in the PWM Analyzer, the watchdog resets the internal dead time counters if no valid PWM transitions occur within a specified interval, thereby preventing outdated or incorrect timing data from reaching the processing system.

The control board incorporates an OVC protection module that detects overcurrent and short-circuit conditions. This module disables the PWM outputs upon identification of a fault. Following the successful testing of the PWM and CurrentSense pathways, the next objective is to verify the protection logic’s functionality under fault scenarios. The following section details the OVC verification process.

4.3 OVC Verification

The OVC module continuously monitors phase currents by capturing current-sense values generated by the SPI module. These values are compared to configurable thresholds, which are adjustable via the PS of the control board (RPU). Upon detection of an overcurrent or

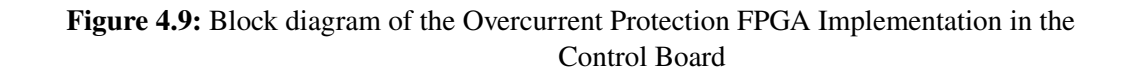


Figure 4.9 presents the OVC Protection module in conjunction with the PWM OFF module as implemented on the control board. The PWM OFF module functions as an error-signal detection system that tristates the PWM output buffers when any of the following five error conditions are detected:

- The default configuration parameters for the overcurrent protection system are as follows:

- `ovc_offset` = 0x800 $\approx$ 0A
- `ovc_low_limit` = 0x650 $\approx \pm 65$ A (`ovc_event` occurs)
- `ovc_timer` = 0x280 $\approx 10 \mu$ s (running with 65MHz $\rightarrow$ waiting on `ovc_low_limit` before error indication)
- `ovc_high_limit` = 0x7CE $\approx \pm 80.5$ A (`ovc_event` occurs)

4.3.1 Block Design of OVC Verification

Verification of the OVC protection module utilizes the setup shown in Figure 4.10. The testing methodology closely follows the procedure established for validating the Current Sense block, allowing for reuse of the DAC-based signal generation and UART communication interface with only minor modifications. In this test, a constant digital value is loaded into the DAC, which is subsequently converted into an analog signal representing the measured phase current and applied to the control board.

If the simulated current exceeds the OVC threshold of approximately ± 65 A, the protection logic activates the overcurrent indicator for the corresponding phase. The system monitors the duration for which the signal remains above this threshold. Should the overcurrent condition persist beyond the 10μ s internal timeout, the module identifies it as a genuine fault and asserts the `ovc_event` output. The design also provides rapid protection against severe faults: if the sensed value exceeds the upper ± 80.5 A boundary, the module immediately classifies the event as a short-circuit condition. In this scenario, the short-circuit flag is set immediately, and `ovc_event` is triggered without delay.

In addition to transmitting the OVC_Verification control commands (`ovc_u_a1`, `ovc_v_a1`, `ovc_w_a1`), the FSM in the Verification MPSoC writes to the DAC. The OVC module on the control board then captures the following signals and transmits them via UART back to MPSoC for further processing in the PS, using a two-byte format:

- Byte 0: `ovc_status[0:5]` (active high)
 - $\rightarrow$ Bit 0 – overcurrent on phase U
 - $\rightarrow$ Bit 1 – short circuit on phase U

- Bit 2 – overcurrent on phase V
- Bit 3 – short circuit on phase V
- Bit 4 – overcurrent on phase W
- Bit 5 – short circuit on phase W

- Byte 1: Bit 0 - ovc_event - overcurrent error indication (active high)

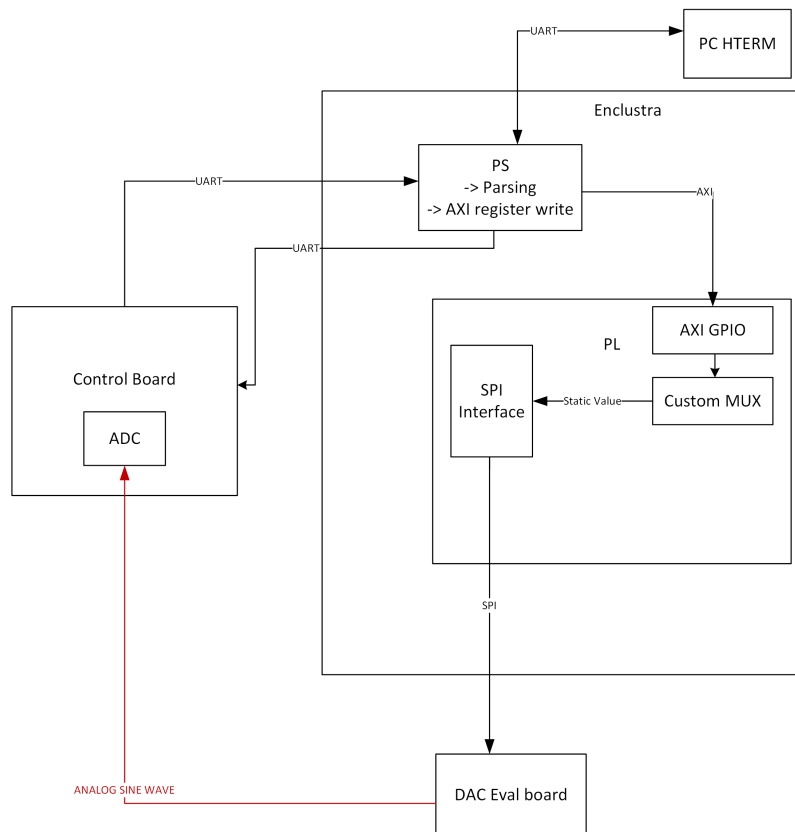


Figure 4.10: Block Design of the Overcurrent Verification setup: Note that apart from the DDFS, the same analog path is reused for OVC Verification

Previous sections have examined the functional blocks within the PL and the hardware mechanisms responsible for signal generation, measurement, and real-time control. The following section addresses the PS, which executes software to supervise and coordinate these hardware modules. Within the PS, a dedicated Verification Controller finite state machine manages user interaction via UART, facilitates communication with PL blocks,

and validates feedback from the PL modules. The PS is also responsible for higher-level tasks such as FFT processing, frequency estimation, and duty-cycle calculation, which constitute the core of the verification framework. This co-design of software and hardware results in a tightly integrated system, enabling complex processing tasks to operate alongside deterministic logic operations to ensure accurate and reliable system behavior.

4.4 Processing Subsystem application

The servo drive control board FPGA verification application utilizes one of the A53 processor cores from the MPSoC. The verification controller FSM (see figure 4.11) manages both user interaction and validation of feedback from the PL, serving as the backbone of the verification platform.

4.4.1 Communication and Data Handling Layer

The Processing Subsystem employs two distinct UART interfaces (refer figure 3.11) on the Zynq UltraScale+ MPSoC to facilitate communication with external devices and PL. UART0 establishes a connection with the host PC for command transmission, runtime configuration, and reporting of verification results. UART1 serves as a data exchange interface between the verification MPSoC and the servo control board, managing measurement data, trigger signals, and diagnostic responses during validation.

Distinct communication strategies are implemented for each interface to maintain reliability under varying data rates and unpredictable traffic conditions. UART0 utilizes interrupt-driven reception and transmission. Interrupts are triggered by the arrival of new data, timeouts, or the emptying of the transmit FIFO. The interrupt handler processes incoming data, and promptly prepares the UART driver for subsequent messages. This configuration enables non-blocking interaction with the PC terminal, allowing the Verification Controller's FSM to operate independently of user input delays.

UART1 similarly employs interrupt handling, however, all incoming bytes are stored in a software ring buffer to prevent data loss during burst transfers. The UART1 interrupt

service routine copies new data into the circular buffer for later parsing. This method separates rapid data collection from slower processing tasks, ensuring that no data is lost between verification calls.

During system initialization, both UART modules are configured by setting up the XUartPs drivers, specifying the baud rates (115200 Bd for the PC and 921600 Bd for the control board), and registering interrupt handlers with the Generic Interrupt Controller (GIC). Interrupts are configured to trigger on receive-full, timeout, and overrun events, ensuring that the firmware remains informed of critical communication events.

These interfaces collectively serve as the primary communication link for the verification platform, enabling data exchange, and efficient coordination PS, PL, and external control hardware.

4.4.2 Verification Controller fsm

Control commands are transmitted from the user to the system via UART0. These commands specify the verification procedure to be executed. A command parser within the PS interprets each incoming command and initiates the appropriate verification routine. The supported control commands are summarized below:

As shown in Table 4.2, when the FSM receives a control command, it splits the command string at each underscore. If the initial segment corresponds to a motor phase, such as "phu," "phv," or "phw," the FSM transitions to the CurrentSense_Verification state. In this state, the AXI register for the selected phase is updated according to the command. The FSM then examines the subsequent segment of the command. If it identifies "s," the FSM transitions to the CurrentSense_Static state. If it identifies "d," it transitions to the CurrentSense_Dynamic state. In the CurrentSense_Static state, a predetermined static value is transmitted to the DAC to generate a constant analog signal. The system can perform either a single static test or a sequence of tests. For multiple tests, a series of 16-bit values ranging from 0x0000 to 0xFFFF is written to the DAC to simulate various analog conditions. The DAC output is connected to the control board's ADC. The control

board responds to these static analog inputs by providing 12-bit feedback to the verification MPSoC via UART1 (refer to section 4.1).

Table 4.2: Overview of Control Commands and Corresponding Verification Functions

Control Command(s)	Verification Type	Description
phu_s_a1, phv_s_a1, phw_s_a1	Current Sense Static Verification	Instruct the verification controller fsm to perform static current-sense validation for phases U,V and W respectively
phu_d_a1, phv_d_a1, phw_d_a1	Current Sense Dynamic Verification	Performs dynamic current-sense verification for the specified phase.
pwm_v	PWM Module Verification	Triggers the PWM verification sequence. Internally the FSM sends pwm_f_a1 together with the desired frequency value to configure the PWM frequency, and pwm_d_a1 with the corresponding duty-cycle value to adjust the PWM duty cycle.
ovc_u_a1, ovc_v_a1, ovc_w_a1	OVC Module Verification	Triggers over-current (OVC) verification for the requested phase.

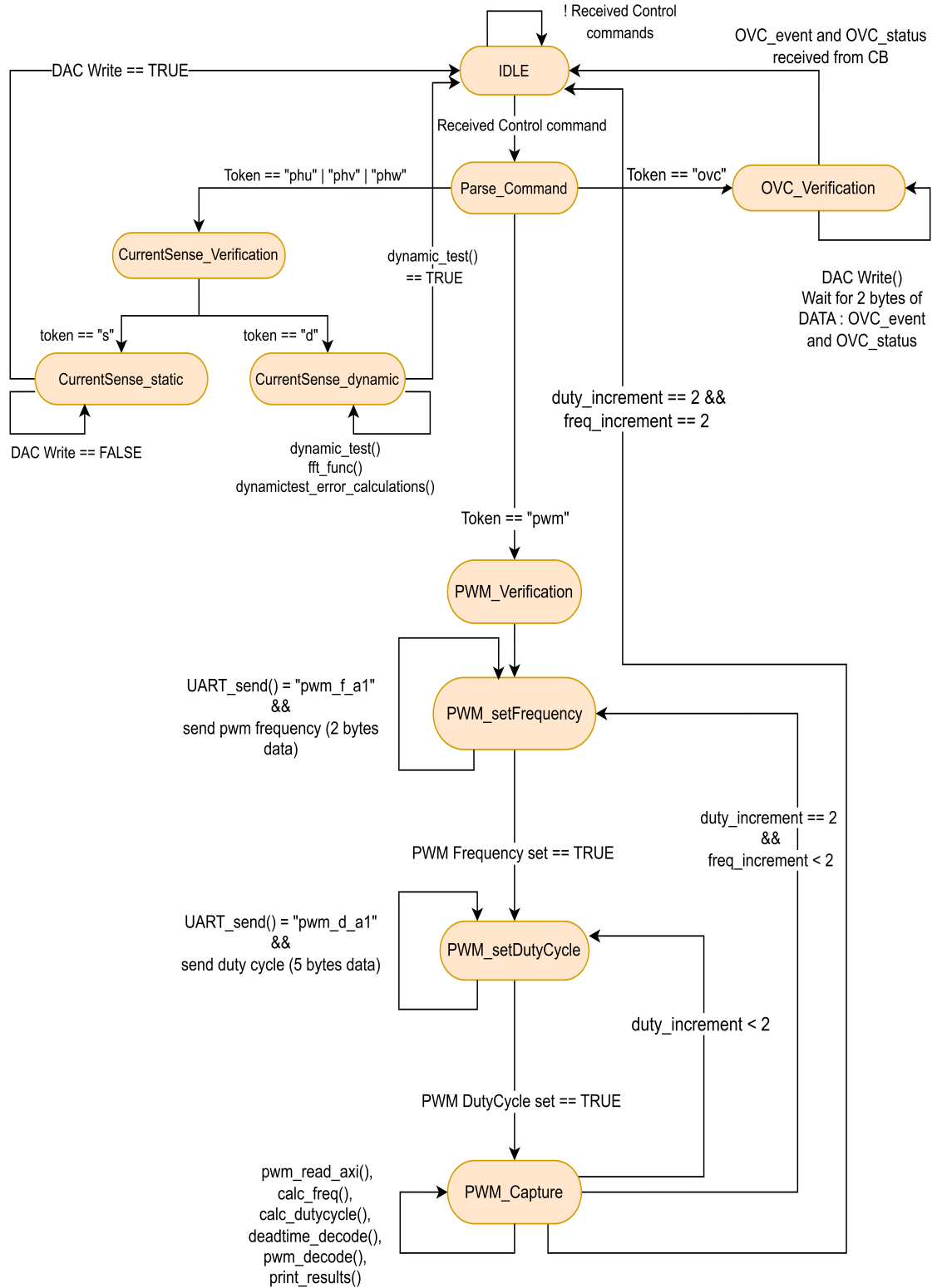


Figure 4.11: Verification Controller FSM

4.4.3 Signal Acquisition and Spectral Analysis

Upon detection of the command token "d," the FSM (figure 4.11) transitions to the CurrentSense.Dynamic state. In this state, a trigger signal instructs the FPGA PL modules to initiate the DDFS module, which generates a 1 kHz sine wave for dynamic verification. Following activation of the DDFS module, the PS awaits the receipt of 4096 samples from the control board. These samples are transmitted over UART1 in 8-bit segments and stored in a ring buffer within the PS for subsequent processing.

The control board transmits 12-bit ADC values, whereas the FPGA DDFS module generates 16-bit digital samples. For comparison, the raw data received via UART must be reconstructed and appropriately scaled. Initially, two 8-bit bytes are combined to form a single 16-bit word using the following expression:

```
1 dynamic_data_buffer[i] = ((uint16_t)data1 << 8) | data2;
```

In this process, the first byte (data1) is shifted left by 8 bits to occupy the higher portion of the word, and the second byte (data2) is combined using a bitwise OR operation. This method reconstructs the original sample transmitted over UART.

Subsequently, the 12-bit ADC values are converted to analog voltage levels, providing a consistent reference for comparison with the DDFS module's 3.3 V output. The conversion is performed using the following formula:

```
1 decData = ((float32_t)hwData / pow(2, 12)) * 3.3;
```

In this context, hwData represents the raw 12-bit ADC reading. Dividing by 2^{12} normalizes the value to a fraction of the full scale, and multiplying by 3.3 V converts it to the actual voltage at the DAC output. Expressing both the control board and FPGA reference signals in voltage units facilitates direct comparison, despite differences in bit resolution.

The voltage data is subsequently processed using a FFT to analyze its frequency content. The resulting spectral peak is compared to the expected 1 kHz sine wave generated by the DDFS module. This comparison verifies the accuracy of the control board's dynamic current-sense response. Utilizing voltage units also mitigates errors arising

from differences in bit resolution, thereby enhancing the reliability and precision of the verification process.

After reconstructing and converting dynamic current-sense samples to voltage values, the system verifies that the control board generates the expected 1 kHz sinusoidal waveform via spectral analysis. This process is managed by two primary functions: **fft_func()**, which computes the frequency spectrum, and **dynamicTestError_calculations()**, which compares the measured waveform to the reference based on frequency, amplitude, and RMS values.

To enable accurate and efficient analysis on the ARM Cortex-A53 core, the system utilizes the CMSIS-DSP library, a signal-processing framework developed by ARM. CMSIS-DSP provides optimized routines for fast FFTs, statistical analysis, vector mathematics, and filtering. The FFT function implemented, **arm_rfft_fast_f32()**, employs pre-computed twiddle factors and radix-based algorithms specifically designed for ARM processors. This configuration reduces computational overhead and supports real-time verification, even with large sample sizes such as 4096 points. Using CMSIS-DSP enables the system to achieve predictable timing, reliable numerical accuracy, and reduced CPU usage compared to a manual FFT implementation.

1. FFT Computation (fft_func())

The **fft_func()** function identifies the primary frequency component in the signal and verifies whether the measured waveform corresponds to the expected 1 kHz output from the DDS module.

```
1 arm_rfft_fast_init_f32(&S, FFT_SIZE);  
2 arm_mean_f32(dynamic_data_decimal_buffer, FFT_SIZE, &mean);
```

The FFT is configured to use 2048 points. Initially, the mean value of the input buffer is calculated and subtracted from each sample. Removal of the DC offset is essential for several reasons:

- It prevents the DC component from dominating the frequency spectrum.
- It reduces spectral leakage within the frequency analysis.

- It ensures accurate identification of the true dominant frequency component.

FFT Execution

The CMSIS function executes a real FFT directly on the input data.

```
1 arm_rfft_fast_f32(&S, dynamic_data_decimal_buffer,  
2 fft_output_buffer, 0);
```

The output is a set of interleaved real and imaginary values, requiring magnitude computation for each FFT bin.

Magnitude Extraction

The magnitude of each frequency component can be determined using the following formula:

$$\text{mag}[k] = \sqrt{(\text{Re}\{X[k]\})^2 + (\text{Im}\{X[k]\})^2}$$

This calculation should be repeated for each bin up to the Nyquist frequency:

```
1 for (int i = 1; i < (FFT_SIZE / 2); i++) {  
2     float32_t real = fft_output_buffer[2 * i];  
3     float32_t imag = fft_output_buffer[2 * i + 1];  
4     magnitude_buffer[i] = sqrtf(real * real + imag * imag);  
5 }
```

Peak Frequency Identification

CMSIS provides a function to efficiently identify the bin with the highest magnitude:

```
1 arm_max_f32(magnitude_buffer, (FFT_SIZE/2) + 1,  
2 &max_val, &max_index);
```

Each bin corresponds to a frequency, which is calculated as follows:

$$\text{bin width} = \frac{F_s}{N}$$

where F_s (Sampling frequency of the Control Board ADC) = 984,615.3846 Hz. The measured frequency can then be determined as follows:

```
1 freq = max_index * bin_width;
```

This calculation yields the primary sinusoidal frequency present in the received signal.

2. Dynamic Error Calculation (dynamicTestError_calculations())

The function determines the fidelity of the measured signal in comparison to the ideal reference sinusoid produced by the DDFS module.

Frequency Error

```
1 error_f = 100/REF_MAX_FREQUENCY *  
2 fabsf(REF_MAX_FREQUENCY - freq);
```

This expression quantifies the percentage deviation of the measured frequency from the expected value of 1 kHz.

Amplitude Error

Amplitude is determined by measuring the peak-to-peak value of the buffered signal. This step also identifies gain-related errors within the control board's sensing path.

```
1 error_amp = 100/REF_PEAK2PEAK_VALUE *  
2 (REF_PEAK2PEAK_VALUE -  
3 peak2peak_f32(dynamic_data_decimal_buffer_copy , FFT_SIZE));
```

RMS Error

The root mean square (RMS) value is calculated using the CMSIS library.

```
1 arm_rms_f32(dynamic_data_decimal_buffer_copy ,  
2 FFT_SIZE, &rms_result);
```

The RMS error is significant because it reflects both the amplitude and the waveform shape.

```
1 error_rms = 100/REF_RMS_VALUE * (REF_RMS_VALUE - rms_result);
```

Pass/Fail Evaluation

The system accepts the measurement if all three errors remain below 5

```
1 if ((error_f<5.0f) && (error_amp<5.0f) && (error_rms<5.0f))  
2 dynamicTestResult_passed();  
3 else  
4 dynamicTestResult_failed();
```

The two functions described above are essential to the dynamic verification process. The `fft_func()` function uses a CMSIS-optimized real FFT to analyze the spectrum and identify the main frequency in the reconstructed voltage samples. The `dynamicTestError_calculations()` function evaluates frequency deviation, amplitude accuracy, and RMS values against reference points. Together, these functions verify that the control board's dynamic current-sense path accurately reproduces the 1 kHz sinusoidal test signal from the DDFS module. CMSIS-DSP ensures strong performance, reliable calculations, and consistent signal processing, which are critical for comprehensive MPSoC-based hardware testing.

4.4.4 Pulse-based Signal Evaluation

When the command parser detects the token "pwm", the fsm transitions from the `Parse_command` state to the `PWM_Verification` state (see figure 4.11). This process verifies the PWM_IP (FPGA) module's ability to generate the PWM signals with specified frequencies and duty cycles. As outlined in the state-machine diagram (figure 4.11), the PWM verification routine employs an automated procedure in which the PS sets three

distinct PWM carrier frequencies: 16 kHz, 8 kHz, and 4 kHz. For each frequency, three specific duty-cycle settings are tested, yielding a total of 9 PWM scenarios. Each scenario is recorded and evaluated by the Verification MPSoC.

The PWM module on the control board generates six outputs corresponding to the high-side and low-side switching signals for each of the three motor phases.

- Phase U: UH (high-side), UL (low-side)
- Phase V: VH (high-side), VL (low-side)
- Phase W: WH (high-side), WL (low-side)

During each test, all six signals operate at the same switching frequency, while their duty cycles vary according to patterns defined by the Processing Subsystem. The PWM_IP Verification is initiated by the PS through the sequential transmission of four UART commands:

1. The first command, `pwm_f_a1`, indicates that the subsequent payload will specify the pwm frequency.
2. The second command transmits the encoded 16-bit frequency value, such as 16000 Hz, 8000 Hz, or 4000 Hz.
3. After the frequency is confirmed, the FSM transitions to the `PWM_setDutyCycle` state. The third command, `pwm_d_a1`, then indicates that the subsequent payload specifies the duty-cycle settings.
4. The fourth command transmits the the duty-cycle settings as a 40-bit value, which includes three duty-cycle values for the high-side switches of phases U, V, and W. The system selects the appropriate duty-cycle pattern from predefined tables, such as `duty_16k[]`, `duty_8k[]`, or `duty_4k[]`, according to the current frequency. After transmitting the duty-cycle values, the system introduces a brief delay using `usleep` to allow the control board's PWM generator to adjust before recording the results.

In the PWM.Capture state (figure 4.11), the PS retrieves PWM measurement results from the FPGA's AXI-based PWM capture IP block. This custom module records the following parameters:

- **The PWM period count** (from which the switching frequency is derived),
- **The ON-time count** (used to calculate the duty cycle),
- **The dead-time counter values** for both High→Low (HL) and Low→High (LH) transitions.

The PS obtains these measurements by accessing specific memory-mapped AXI register offsets corresponding to each phase and switching leg. For example:

- **PHASE_UH_FREQUENCY_DUTYCYCLE** returns a 32-bit word encoding the UH period and duty values.
- **PHASE_U_DEADTIME** provides the dead-time timestamps for phase U.

The PS application decodes these raw register values. The lower 16 bits represent the PWM period (T_{PWM}), while the upper 16 bits contain the duty-cycle count (T_{ON}). The PWM frequency and duty cycle are calculated as follows:

$$f_{PWM} = \frac{192MHz}{T_{PWM}}, \quad Duty(\%) = \frac{T_{ON}}{T_{PWM}} \times 100$$

The 192 MHz frequency reference corresponds to the clock that drives the PWM capture logic. This reference enables accurate conversion of clock counts to real signal characteristics.

Dead-time values are also converted to microseconds:

$$Deadtime[\mu s] = \frac{DT_count}{192MHz} \times 10^6$$

This conversion ensures that the dead-time behavior of the control board's gate-drive logic is measured accurately and can be compared across test runs. After decoding measurements for all six outputs (UH, UL, VH, VL, WH, WL), the PS generates a structured text report. This report is transmitted over UART and includes the following information:

- PWM frequency of each phase's high-side and low-side switch,
- Duty cycle accuracy,
- High→Low and Low→High dead-time durations.

The FSM increments the duty-cycle index. After completing all three duty-cycle patterns for the current frequency, the system advances to the next frequency. Upon completion of all nine scenarios (three frequencies and three duty cycles), the FSM returns to the Idle state and awaits the next user command.

This automated verification process ensures that the control board's PWM_IP FPGA module meets the stringent timing requirements of motor-control applications. By evaluating all switching legs (UH/UL, VH/VL, WH/WL), the system verifies Frequency stability across all phases, Duty-cycle accuracy and linearity, Dead-time symmetry and correctness, Compliance with expected motor-control pulse patterns. The Verification MPSoC hardware directly captures the PWM signals, and the PS software decodes them. This configuration provides independent, high-resolution measurements of PWM behavior.

4.4.5 Protection Event Monitoring

The OVC Protection module verification ensures that the Control Board MPSoC correctly detects and reports the over-current events under test conditions. The verification procedure shares conceptual similarities with the static path used in the Current Sense module. A single static value is transmitted to the DAC path of the Verification MPSoC, activating the OVC FPGA module on the Control Board. In contrast to current measurement tests, the OVC module provides only event and status information rather than data samples.

Upon detection of the "ovc" token by the command parser, the Processing Subsystem FSM transitions from the Parse_command state to the OVC_Verification state (refer to the verification controller state machine diagram 4.11) . Within this state, the Verification MPSoC writes a user-defined synthetic current value to the DAC and Subsequently, the OVC status and event codes are returned from the control board to the Verification MPSoC via UART1, enabling comprehensive validation of the hardware module.

When the FSM enters the OVC_Verification state, the PS requests a 16-bit hexadecimal current value from the user. To ensure input validity, the buffer is examined using a dedicated function:

```
1 bool is_valid_16bit_hex(const char *str);
```

This function verifies the following conditions:

- The input adheres to the format 0xNNNN.
- Only valid hexadecimal characters are present.
- If the input is invalid, the PS instructs the user to restart the test.

Following validation, the hexadecimal string is converted to a 16-bit integer using the following method:

```
1 hexValue = (uint16_t)strtoul(buffer, NULL, 16);
```

The PS subsequently transmits this value to the PL of the control board (OVC module):

```
1 ovc_axi_staticData_write(hexValue);  
2 ovc_axi_startOp_write();
```

This operation prompts the OVC module on the control board to verify whether the injected current exceeds its configured threshold (refer section 4.3).

After initiating the OVC operation, the PS waits for a response from the control board via UART1. The OVC logic returns two bytes of data, OVC_event and OVC_status. Upon receipt of the two expected bytes, the PS reads and displays them as output.

The OVC module enables the control board to reliably detect faults that could potentially damage motor drives or power stages. The verification flow implemented in the MPSoC offers the following capabilities:

- Deterministic injection of synthetic current values,
- Validation of thresholds against the control board's configured over-current level,
- Testing of event-flag correctness, ensuring proper assertion and deassertion behavior,
- Validation of the communication path between the control board and the verification MPSoC.

This setup facilitates early detection of configuration errors, calibration issues, or communication faults before deploying the control board into an operational motor. Integration of the OVC test with the central FSM and UART command interface automates and standardizes the test suite, supporting regression testing during firmware updates or hardware modifications.

5 Results and Validation

This chapter presents the results of verification tests conducted on the Control Board (CB). The primary objective is to analyze the system's feedback signals and compare them to the expected reference values. This comparison assesses whether the hardware and application software operate correctly and reliably.

5.1 Current Sense Verification Results

The Current Sense Verification evaluates the accuracy and consistency of the current measurement path on the Control Board. This evaluation involves analyzing digital feedback from the current-sensing circuit during both static and dynamic input tests. Reference values are transmitted to the DAC, and feedback from the CB is collected. The results are processed and plotted in MATLAB to verify that the outputs align with expectations. The following sections present the results from the static and dynamic tests.

5.1.1 Static Test Results

The Current Sense Static Verification assesses the sensing system's performance under constant input conditions. The feedback from the CB is provided as a 12-bit digital value, while the reference values sent to the DAC are 16-bit. MATLAB is utilized to scale, analyze, and verify these values.

During the static test, six distinct 16-bit values (0x0000, 0x3333, 0x6666, 0x9999, 0xCCCC, and 0xFFFF) were sequentially transmitted to the DAC, each establishing a fixed current reference. The corresponding 12-bit digital feedback from the CB was recorded for each value. These results were plotted in MATLAB to illustrate the relationship between the measured outputs and the reference values. As shown in Figure 5.1, the results exhibit a linear trend from 0x000 to 0xFFFF, demonstrating that the current-sensing path performs as expected across various static inputs.

5.1 Current Sense Verification Results

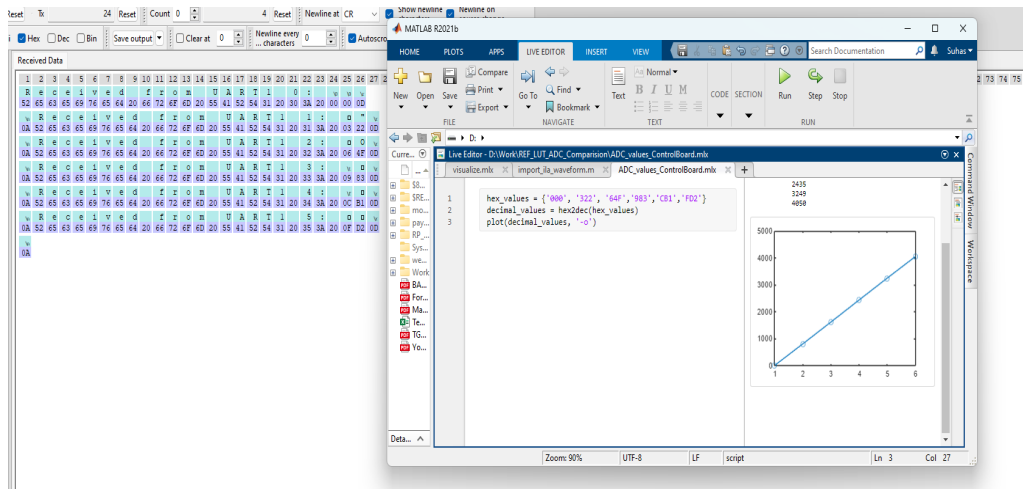


Figure 5.1: Current Sense Static Verification Results

5.1.2 Dynamic Test Results

This test evaluates the current-sensing chain's response to dynamic changes by comparing the frequency and waveform of feedback from the CB. The CB transmits sampled feedback to the Enclustra module through a 4096-byte FIFO, with each 8-bit word representing a segment of a measurement. These 4096 bytes contain 2048 distinct 12-bit measurement values, transmitted in a packed format with least- and most-significant bytes for each sample.

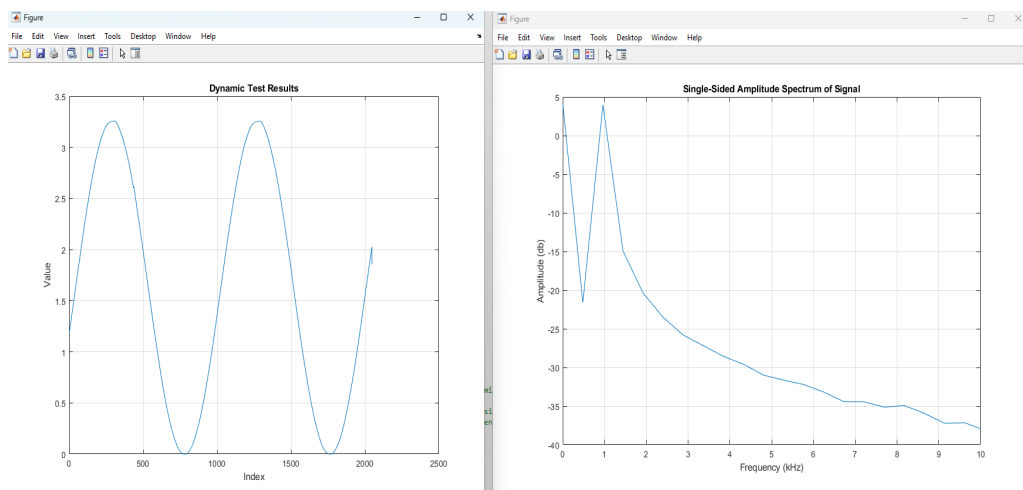


Figure 5.2: Reconstructed time domain representation and FFT of the feedback signal in matlab

Upon receipt, the PS analyses by applying FFT and error values are calculated relative to the reference signal as mentioned in the section 4.4.3. The accompanying figure 5.2 presents the time-domain reconstruction of the received waveform, generated in MATLAB. The results demonstrate that the CB accurately captures the dynamic current waveform and that the Enclustra PS preserves the essential frequency content required for correct system operation.

5.1.3 Reference Values vs measured Values

The figure 5.3 compares the converted voltage values of the reference signal with those from the CB during the static test. Digital values were converted to voltage using the method described in Section 4.4.3. The plot demonstrates that the CB feedback closely matches the reference signal, with sample points aligning as anticipated. These results confirm that the current sensing path accurately reproduces the static input across the tested range.

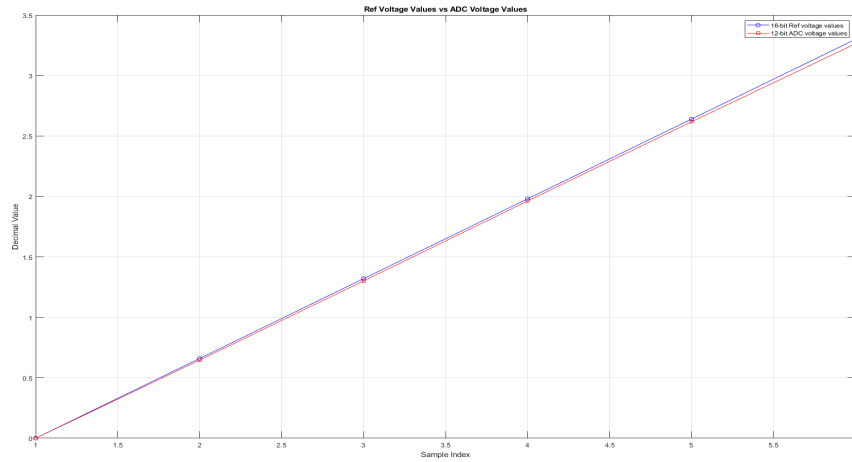


Figure 5.3: Static Reference Voltage Values VS CB feedback Voltage Values

Upon completion of the Current Sense Dynamic Verification, the frequency obtained from the CB feedback and the associated error calculations relative to the reference signal are recorded in an output file to ensure documentation and traceability. The analysis results are presented in the accompanying figure 5.4. When applying a 5% error threshold, all calculated metrics, including the derived frequency, remain within acceptable limits.

5.1 Current Sense Verification Results

```
Phase V - dynamic test - Frequency : 961.538Hz Freq Error: 3.846% Amp Error: 1.243% RMS Error: 2.939% TEST PASSED
Phase U - dynamic test - Frequency : 961.538Hz Freq Error: 3.846% Amp Error: 1.268% RMS Error: 0.651% TEST PASSED
Phase W - dynamic test - Frequency : 961.538Hz Freq Error: 3.846% Amp Error: 1.219% RMS Error: 0.417% TEST PASSED
Phase U - dynamic test - Frequency : 961.538Hz Freq Error: 3.846% Amp Error: 1.243% RMS Error: 2.678% TEST PASSED
Phase W - dynamic test - Frequency : 961.538Hz Freq Error: 3.846% Amp Error: 1.194% RMS Error: 0.770% TEST PASSED
Phase W - dynamic test - Frequency : 961.538Hz Freq Error: 3.846% Amp Error: 1.194% RMS Error: 0.359% TEST PASSED
Phase V - dynamic test - Frequency : 961.538Hz Freq Error: 3.846% Amp Error: 1.243% RMS Error: 0.497% TEST PASSED
Phase U - dynamic test - Frequency : 961.538Hz Freq Error: 3.846% Amp Error: 1.243% RMS Error: 2.499% TEST PASSED
Phase V - dynamic test - Frequency : 961.538Hz Freq Error: 3.846% Amp Error: 1.243% RMS Error: 1.542% TEST PASSED
Phase W - dynamic test - Frequency : 961.538Hz Freq Error: 3.846% Amp Error: 1.194% RMS Error: 0.181% TEST PASSED
Phase U - dynamic test - Frequency : 961.538Hz Freq Error: 3.846% Amp Error: 1.268% RMS Error: 2.352% TEST PASSED
Phase V - dynamic test - Frequency : 961.538Hz Freq Error: 3.846% Amp Error: 1.219% RMS Error: 2.478% TEST PASSED
Phase W - dynamic test - Frequency : 961.538Hz Freq Error: 3.846% Amp Error: 1.194% RMS Error: 2.908% TEST PASSED
Phase W - dynamic test - Frequency : 961.538Hz Freq Error: 3.846% Amp Error: 1.194% RMS Error: 0.752% TEST PASSED
Phase V - dynamic test - Frequency : 961.538Hz Freq Error: 3.846% Amp Error: 1.243% RMS Error: 0.442% TEST PASSED
Phase U - dynamic test - Frequency : 961.538Hz Freq Error: 3.846% Amp Error: 1.268% RMS Error: 1.310% TEST PASSED
Phase V - dynamic test - Frequency : 961.538Hz Freq Error: 3.846% Amp Error: 1.219% RMS Error: 2.707% TEST PASSED
Phase U - dynamic test - Frequency : 961.538Hz Freq Error: 3.846% Amp Error: 1.243% RMS Error: 2.899% TEST PASSED
Phase W - dynamic test - Frequency : 961.538Hz Freq Error: 3.846% Amp Error: 1.219% RMS Error: 1.752% TEST PASSED
Phase W - dynamic test - Frequency : 961.538Hz Freq Error: 3.846% Amp Error: 1.194% RMS Error: 0.062% TEST PASSED
Phase U - dynamic test - Frequency : 961.538Hz Freq Error: 3.846% Amp Error: 1.243% RMS Error: 1.734% TEST PASSED
Phase V - dynamic test - Frequency : 961.538Hz Freq Error: 3.846% Amp Error: 1.219% RMS Error: 1.137% TEST PASSED
Phase V - dynamic test - Frequency : 961.538Hz Freq Error: 3.846% Amp Error: 1.219% RMS Error: 0.343% TEST PASSED
Phase U - dynamic test - Frequency : 961.538Hz Freq Error: 3.846% Amp Error: 1.243% RMS Error: 2.856% TEST PASSED
Phase W - dynamic test - Frequency : 961.538Hz Freq Error: 3.846% Amp Error: 1.194% RMS Error: 2.284% TEST PASSED
```

Figure 5.4: Current Sense Dynamic Test Results

Consequently, the verification concludes with a “TEST PASSED” status, as indicated in the figure 5.4, thereby confirming that the dynamic response of the sensing path satisfies the specified performance criteria.

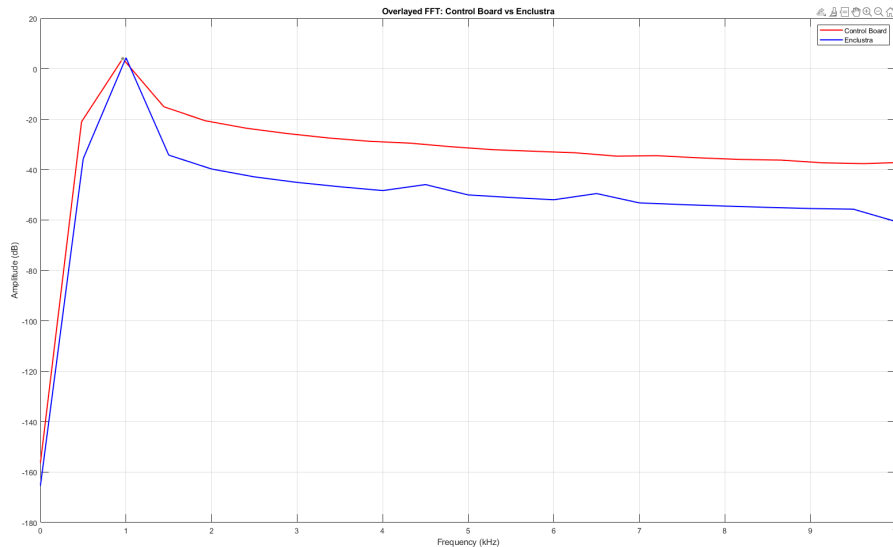


Figure 5.5: FFT comparison of the reference 1 kHz digital signal and the feedback received from the Control Board.

The FFTs of both the CB feedback signal and the original 1 kHz digital reference signal were calculated and analyzed in MATLAB. The resulting spectra are presented in the accompanying figure 5.5. This MATLAB analysis corroborates the frequency

characteristics observed in the experimental results, thereby supporting the validity and reliability of the Dynamic Current Sense Verification. The close correspondence between the reference spectrum and the measured feedback spectrum demonstrates that the sensing path maintains the expected dynamic behavior.

5.2 PWM Verification results

The figure 5.6 presents a portion of the PWM verification results. During this process, the PWM frequency and duty cycle are transmitted to the CB, which generates the corresponding PWM signals and returns them to the verification platform. The PS evaluates these signals using the method outlined in Section 4.4.4. For each `pwm_v` command, three PWM channels corresponding to the three motor phases are tested with three distinct duty-cycle settings, resulting in nine internal test sequences. The measured parameters from each sequence, including frequency, duty cycle, and dead time, are recorded in an output file.

```
Please send the command to UART0 (e.g., phu_s): <\r><\n>
Received command: pwm_v<\r><\n>
PWM frequency phase U High: 16000.000Hz and duty cycle: 48.400% deadtime: 1.000us<\n>
PWM frequency phase U Low: 16000.000Hz and duty cycle: 48.400% deadtime: 0.994us<\n>
PWM frequency phase V High: 16000.000Hz and duty cycle: 48.400% deadtime: 1.000us<\n>
PWM frequency phase V Low: 16000.000Hz and duty cycle: 48.408% deadtime: 0.994us<\n>
PWM frequency phase W High: 16000.000Hz and duty cycle: 48.400% deadtime: 1.000us<\n>
PWM frequency phase W low: 16000.000Hz and duty cycle: 48.408% deadtime: 0.994us<\n>

PWM frequency phase U High: 8000.000Hz and duty cycle: 39.200% deadtime: 1.005us<\n>
PWM frequency phase U Low: 8000.000Hz and duty cycle: 59.200% deadtime: 0.994us<\n>
PWM frequency phase V High: 7999.666Hz and duty cycle: 69.192% deadtime: 1.000us<\n>
PWM frequency phase V Low: 8000.000Hz and duty cycle: 29.204% deadtime: 1.000us<\n>
PWM frequency phase W High: 8000.000Hz and duty cycle: 24.200% deadtime: 1.000us<\n>
PWM frequency phase W low: 8000.000Hz and duty cycle: 74.199% deadtime: 1.000us<\n>

PWM frequency phase U High: 4000.000Hz and duty cycle: 34.597% deadtime: 1.010us<\n>
PWM frequency phase U Low: 4000.000Hz and duty cycle: 64.600% deadtime: 0.994us<\n>
PWM frequency phase V High: 4000.000Hz and duty cycle: 14.599% deadtime: 1.000us<\n>
PWM frequency phase V Low: 4000.000Hz and duty cycle: 84.599% deadtime: 1.000us<\n>
PWM frequency phase W High: 4000.000Hz and duty cycle: 74.595% deadtime: 1.010us<\n>
PWM frequency phase W low: 4000.000Hz and duty cycle: 24.600% deadtime: 1.000us<\n>
```

Figure 5.6: PWM Verification Results

The figure displays results from three representative test sequences. In the first sequence, the PWM frequency is configured to 16 kHz with a 50% duty cycle for all phases, and

the dead time is set to 1 μ s. In the second sequence, the PWM frequency is 8 kHz, and the duty cycles differ by phase (for example, UH/UL at 40%/60%, VH/VL at 70%/30%, and WH/WL at 30%/70%). In the third sequence, the PWM frequency is set to 4 kHz and likewise variable phases.

Across all sequences, the measured duty-cycle values exhibit minor, consistent deviations from the intended settings. This outcome is anticipated, as the 1 μ s dead time reduces the available high-pulse width within each PWM period. The measured values presented in the figure 5.6 demonstrate this effect and confirm that the verification platform accurately records the PWM characteristics.

5.3 OVC Verification Results

The Control Board outputs a 2-byte value for each static current level configured by the verification MPSOC. These bytes represent over-current event and status information, as detailed in Section 4.3. The accompanying figure illustrates the system's response in two representative test cases. When the current remains within the OVC limits, the Control Board returns "00", signifying the absence of any overcurrent event or status condition.

In the subsequent test, a value of 0x0000 is applied, which falls outside the permissible OVC threshold. In this scenario, the Control Board returns "1F". The first byte signifies that the `ovc_event` flag is set, while the second byte indicates that all bits in the `ovc_status` field are high. This outcome demonstrates that all three phases are experiencing a short-circuit or fault condition. These results confirm that the OVC detection functionality on the Control Board operates as intended for out-of-range current inputs.

```
Please send the command to UART0 (e.g., phu_s): <\r><\n>
Received command: ovc_u_a1<\r><\n>
Please send the 16 bit hexadecimal CurrentValue (0x0-f/0x0-F) to receive the OVC status and event valuesReceived value is : 0x9999<\n>
Valid Current (hexa)Value<\n>
Received from UART1: <\0><\0><\r><\n>
Please send the command to UART0 (e.g., phu_s): <\r><\n>
Received command: ovc_v_a1<\r><\n>
Please send the 16 bit hexadecimal CurrentValue (0x0-f/0x0-F) to receive the OVC status and event valuesReceived value is : 0x0000<\n>
Valid Current (hexa)Value<\n>
Received from UART1: <1><f><\r><\n>
Please send the command to UART0 (e.g., phu_s): <\r><\n>
Received command: ovc_v_a1<\r><\n>
Please send the 16 bit hexadecimal CurrentValue (0x0-f/0x0-F) to receive the OVC status and event valuesReceived value is : 0x9999<\n>
Valid Current (hexa)Value<\n>
Received from UART1: <\0><\0><\r><\n>
Please send the command to UART0 (e.g., phu_s): <\r><\n>
```

Figure 5.7: Overcurrent module Verification Results

6 Conclusion

This chapter reviews the principal findings of the research and evaluates the effectiveness of the proposed FPGA Verification Platform. It summarizes the key contributions, examines how the proof-of-concept addresses the initial challenges outlined in the thesis, and discusses the implications for servo-drive FPGA validation. Additionally, the chapter proposes potential extensions and improvements to the platform, highlighting its capacity to facilitate future advancements in industrial motor-control verification.

6.1 Summary

This thesis presents the design, development, and validation of a dedicated FPGA Verification Platform for servo-drive control boards utilizing the Zynq UltraScale+ MPSoC architecture. The growing complexity of FPGA modules in industrial motor control, along with limited test coverage in current verification methodologies and the inherent shortcomings of simulation-based approaches, motivated this research. Simulation methods often do not capture real-world timing, analog feedback, or module interactions, thus, in-system validation is essential to ensure reliable performance.

To address these challenges, this thesis introduces a Proof-of-Concept FPGA-based verification framework. The system generates realistic input signals, executes automated tests, and evaluates device responses within a closed-loop configuration. Operating independently from the complete motor-control system, it enables black-box testing of individual modules. This approach enhances test coverage, simplifies debugging, and reduces the need for additional hardware.

A comprehensive technical analysis of the MPSoC architecture, clocking strategies, verification methodologies, and AMBA AXI communication protocols formed the foundation for this research. Based on these considerations, three critical FPGA modules from the servo-drive control board were selected for verification - Current Sense SPI path, Over-Current Protection module, and PWM generation module.

For each module, dedicated systems were developed to generate test signals, measure outcomes, and analyze data. The Current Sense verification employed both static and dynamic tests using a DAC80508 analog signal, Processing subsystem for FFT analysis, and MATLAB for output validation. The PWM verification assessed frequency, duty cycle, and dead time with high precision. The OVC verification evaluated event codes and protection behavior across various programmed fault thresholds.

Experimental results demonstrated that all tested modules responded appropriately to synthetic inputs. Measurement errors remained within specified limits, and each test concluded with a "TEST PASSED" outcome. The platform successfully replicated key characteristics of motor-control signals, verified the correct operation of the protection logic, and demonstrated strong agreement between the reference and measured values.

The proposed verification platform addresses the primary challenges outlined in the section 1.3 (Problem Statement) . Automated test execution and a standalone MPSoC-based configuration minimize manual intervention and eliminate the need for repeated reconfiguration, thereby increasing automation. The platform can generate both static and dynamic input signals without physical motors, demonstrating that realistic operating conditions can be fully emulated during FPGA verification. By providing each module with an independent test system, the platform enables genuine module-level validation and eliminates the interdependence among FPGA modules, such as the need for a PWM module during Current Sense Verification. Digital measurement techniques, including FFT-based analysis and precise signal verification, replace manual estimation, thereby improving accuracy and repeatability. Experimental results indicate that this approach overcomes the principal limitations of previous methods and provides a more scalable, flexible, and reliable solution for FPGA module verification.

In summary, the verification platform demonstrates that a low-power, fully programmable, MPSoC-based FPGA verification approach can effectively support both analog and digital testing. This Proof-of-Concept establishes a robust foundation for the development of future scalable verification systems in industrial servo-drive applications.

6.2 Future Work

As servo-drive control systems become more complex, the proposed FPGA verification platform can be further enhanced and expanded. Future developments should prioritize improved communication, broader verification coverage, and enhanced usability through increased automation.

A significant upgrade for the platform involves replacing the current UART communication with an Ethernet-based interface. This modification would enable faster data transfer, reduced latency, and more reliable connections between the PC and the Verification MPSoC, particularly when handling large datasets or real-time measurements.

The platform may also be extended to verify additional FPGA modules on the control board. Future enhancements should include encoder interfaces, brake-control logic, analog measurement paths, Ethernet TSN features, and digital I/O modules. Incorporating these elements would facilitate more comprehensive board-level verification.

Subsequent versions of the platform may incorporate FPGA-accelerated HIL models to simulate complete motor-control systems or power electronics. This approach would enable realistic closed-loop testing without the need for physical motors or power hardware, aligning the proof of concept with contemporary HIL and FIL verification methodologies.

Usability may be enhanced by developing a high-level software interface for automated testing, data logging, and result visualization. The addition of dashboards and structured reports would streamline verification processes and improve accessibility for larger engineering teams.

Integrating the verification platform into continuous integration and continuous deployment (CI/CD) pipelines would enable automated regression testing and hardware validation throughout FPGA development. This integration would reduce development risks and support sustained design reliability.

In conclusion, these enhancements have the potential to transform the current proof-of-concept into a robust, scalable, automated, and industry-ready FPGA verification platform suitable for modern servo-drive control systems.

Part II

Appendix

A 16 kHz PWM With Variable Duty Cycles

An example demonstrates the generation of a 16 kHz PWM signal with duty cycles of 50% for phase U, 30% for phase V, and 80% for phase W. The FPGA on the control board operates at a carrier frequency of 64 MHz.

$$\text{pwm_compare_values [DEC]} = \frac{64 \text{ MHz}}{2 \times 16 \text{ kHz}} = 2000$$

$$50\% \text{ Duty Cycle for Phase U} = \frac{50}{100} \times 2000 = 1000$$

$$\text{In HEX} = 0x03E8$$

$$\text{In Binary (13 bits)} = 0001111101000$$

$$30\% \text{ Duty Cycle for Phase V} = \frac{30}{100} \times 2000 = 600$$

$$\text{In HEX} = 0x0258$$

$$\text{In Binary (13 bits)} = 000100101000$$

$$80\% \text{ Duty Cycle for Phase W} = \frac{80}{100} \times 2000 = 1600$$

$$\text{In HEX} = 0x0640$$

$$\text{In Binary (13 bits)} = 001100100000$$

$$\text{Now U/V/W} = 0000111110100000010010110000011001000000$$

$$\text{In HEX} = 0x0FA04B0640$$

The calculated values produce six PWM signals with the designated duty cycles. Figure 4.5 presents the resulting PWM waveforms corresponding to these parameters.

Bibliography

- [1] AMD: *Vivado Design Suite User Guide: Logic Simulation (UG900)*, October 2023. <https://docs.amd.com/r/en-US/ug900-vivado-logic-simulation/Simulation-Flow>
- [2] AMD: Zynq UltraScale+ Overview (DS891). (2024). <https://docs.amd.com/v/u/en-US/ds891-zynq-ultrascale-plus-overview>
- [3] ARM: Learn the architecture - An introduction to AMBA AXI. In: *arm White Paper* (2020-2022). <https://documentation-service.arm.com/static/6560cd802c8b3557fee70a89>
- [4] ARM: AMBA AXI Protocol Specification. (2025). <https://developer.arm.com/documentation/ih10022/latest>
- [5] ARUNA JAYASENA, Prabhat M.: HIVE: Scalable Hardware-Firmware Co-Verification Using Scenario-Based Decomposition and Automated Hint Extraction. In: *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 43, Issue: 10, October 2024 (2024), April, 3278 - 3291. <https://ieeexplore.ieee.org/document/10488040>
- [6] BOULDIN, David: FPGA Design Verification: Techniques for Creating a Fully Functional Design. (2006), May. <https://web.eecs.utk.edu/~dbouldin/protected/verification-synplicity.pdf>
- [7] CHHABRA, Atin: Understanding Servo Motors And Servo Drives. In: *Schneider Electric Blog* (2019), March. <https://blog.se.com/industry/machine-and-process-management/2019/03/07/understanding-servo-motors-and-servo-drives/>
- [8] CUI, Xiaofang: Research and Application of FPGA Function Verification Methods. In: *Academic Journal of Science and Technology* 9 (2024), January, Nr. 1, 30–32. <https://drpress.org/ojs/index.php/ajst/article/view/16289/16336>

- [9] DEBRECENI, Tibor ; KÖKÉNYESI, Tamás ; SÜTŐ, Zoltán ; VARJÁSI, István: FPGA-based Real-Time Hardware-In-the-Loop Simulator of a Mini Solar Power Station. (2014). <https://ieeexplore.ieee.org/document/6850408>
- [10] DUNG, Nguyen Thi N.: An Overview on Electric Motors: Classification, Control and Applications. In: *International Journal of Research and Scientific Innovation* 11 (2024), November, Nr. 11, 318–333. <https://rsisinternational.org/journals/ijrsi/digital-library/volume-11-issue-11/318-333.pdf>
- [11] ELECTRIC, Schneider: Lexium62 Hardware Guide. (2021), February. https://download.schneider-electric.com/files?p_Doc_Ref=EI00000003738
- [12] ELECTRIC, Schneider: Lexium 32 Servo Drives & Motors. (2025). <https://www.se.com/il/en/product-range/2302-lexium-32-motors/>
- [13] ELECTRONICS, Popular: Popular Electronics, April 1959. In: *Popular Electronics* 11 (1959), April, Nr. 4. <https://www.worldradiohistory.com/Archive-Poptronics/50s/59/Pop-1959-04.pdf>
- [14] ENCLUSTRA: Mercury XU5 SoC Module User Manual. (2019), jul. <https://mm.digikey.com/Volume0/opasdata/d220001/medias/docus/6448/MercuryXU5UserManualV07.pdf>
- [15] ENCLUSTRA: Mercury+ PE1 Base Board User Manual. (2020), may. <https://www.rxelectronics.si/datasheet/b1/me-pe1-400-w-r4.6.pdf>
- [16] GNUSIN, Alexander: FPGA-based Clock Domain Crossing Validation for Safety-Critical Designs. In: *DVCon Proceedings* (2022). <https://dvcon-proceedings.org/wp-content/uploads/fpga-based-clock-domain-crossing-validation-for-safety-critical-designs.pdf>
- [17] GONG, Lingkan: *Simulation-based Functional Verification of Dynamically Reconfigurable FPGA-based Systems*, University of New South Wales, Diss., July 2013. <https://cgi.cse.unsw.edu.au/~odiessel/papers/phdgong.pdf>

- [18] INC., Xilinx: Xilinx UltraScale MPSoC Architecture Backgrounder. In: *Xilinx White Paper* (2015), July. https://www.xilinx.com/publications/prod_mktg/ultrascale-mpsoc-architecture-bacgrounder.pdf
- [19] INC., Xilinx: Vivado AXI Reference Guide (UG1037). In: *Xilinx Technical Documentation* (2017). <https://docs.amd.com/v/u/en-US/ug1037-vivado-axi-reference-guide>
- [20] INC., Xilinx: System Integrated Logic Analyzer (System ILA) Product Guide (PG261). (2021), February. <https://docs.amd.com/v/u/en-US/pg261-system-ila>
- [21] INC., Xilinx: Clocking Wizard LogiCORE IP Product Guide (PG065) - 6.0. In: *Product guide* (2022). <https://docs.amd.com/r/en-US/pg065-clk-wiz>
- [22] INSTRUMENTS, Texas: DAC80508-04 Evaluation Module User's Guide. , nov, Nr. SLAU920. <https://www.ti.com/lit/pdf/slau920>
- [23] INSTRUMENTS, Texas: DAC80508 – Octal 16-, 14-, 12-Bit SPI Voltage-Output DAC with Internal Reference (Datasheet, Rev. D). (2018). <https://www.ti.com/lit/ds/symlink/dac80508.pdf>
- [24] KARTHIK, S.: Optimization for Behavioral/RTL Simulation. (1993), January. <https://ieeexplore.ieee.org/document/669702>
- [25] KÖKÉNYESI, T.: FPGA-synthesizable Filter Model Based Bitstream DAC for Hardware-in-the-Loop Simulators. In: *IMEKO TC4 International Symposium Proceedings* (2016). <https://www.imeko.org/publications/tc4-2016/IMEKO-TC4-2016-38.pdf>
- [26] LU, Bin: A Low-Cost Real-Time Hardware-in-the-Loop Testing Approach of Power Electronics Controls. (2007), April. <https://ieeexplore.ieee.org/document/4132974>
- [27] ROGERS, Paul ; KAVASSERI, Rajesh ; SMITH, Scott C.: An FPGA-in-the-loop approach for HDL motor controller verification. (2017). <https://ieeexplore.ieee.org/document/8279774>

- [28] SCHMITT, Alexander ; RICHTER, Jan ; JURKEWITZ, Uli ; BRAUN, Michael: FPGA-based Real-Time Simulation of Nonlinear Permanent Magnet Synchronous Machines for Power Hardware-in-the-Loop Emulation Systems. (2014), October. <https://publikationen.bibliothek.kit.edu/1000046425/3457016>
- [29] SCHNEIDER ELECTRIC: Multi-axis Servo System and Servo Motors for PacDrive3. (2022), March. https://download.schneider-electric.com/files?p_Doc_Ref=DIA7ED2160305EN